CENG 408

Innovative System Design and Development II

2025-2026 Spring

AI Assisted Programming Platform for Education and Assessment

İrfan Gökdeniz YILMAZ 202111038

Emir Tuğberk TOZLU 202011037

Emir KARA 202011068

Ahmet Kerem ÖZTÜRK 202211405

Salih Cihat YALÇIN 202111034

22.05.2026

ABSTRACT

The AI-Assisted Programming Platform for Education and Assessment is a full-stack web application that integrates browser-based programming practice, automated code execution, AI-mediated tutoring, and structured assessment in a single environment. The platform is designed for university-level programming courses where students learn through coding exercises while teachers manage assignments, monitor progress, and evaluate submissions.

The system provides separate interfaces for three roles: students, standard teachers, and a higher-privileged Coordinator Teacher responsible for department-level academic management. Students solve problems in an in-browser editor backed by a Docker-isolated execution sandbox that supports multiple programming languages. They can request help from an AI Mentor that produces Socratic, hint-based guidance rather than direct solutions. The mentor's replies pass through a two-stage validation pipeline — a heuristic policy check followed by a smaller verifier model — that blocks solution leaks and enforces a consistent pedagogical tone. Teachers create and grade assignments, manage a reusable question bank, and apply rubrics, with optional AI-assisted suggestions for problem variations, rubric drafts, and grading scores. All AI suggestions require explicit teacher approval before being applied.

A central concern of the design is academic integrity. The platform enforces this through an Exam Mode that disables AI assistance, locks the student workspace, and gates submissions to a teacher-defined time window. Exam Mode is enforced both in the user interface and at the API layer to prevent client-side bypass. Submissions are versioned, timestamped, and immutable; AI interactions are logged for instructor review; and reference solutions and hidden test cases remain inaccessible to students.

The platform is deployed as a containerized stack on a GPU-equipped virtual machine. The frontend is a React-based single-page application, the backend is a Node.js service exposing a REST and streaming API, persistent data is stored in PostgreSQL with both relational and JSONB columns, and large-language-model inference is served locally to keep student data on-premise. Supporting modules include a tutorial library, feedback flashcards generated from past mistakes, and per-student and class-level analytics.

This report documents the system across four parts: the high-level architecture and design decisions, the implementation details of each module, the testing and validation activities performed against the deployed platform, and a concluding discussion of achievements, limitations, lessons learned, and directions for future work. Together, these sections describe a working prototype that demonstrates how AI assistance can be embedded into programming education without compromising the integrity of assessment.

Table of Contents

System Architecture & Design Decisions

This section details the architectural blueprint of the AI-Assisted Programming Platform, focusing on the centralized management of data flow, modular service breakdown, API design, and deployment strategies required to maintain academic integrity and system scalability.

1. High-Level Architecture

The system follows a **Service-Oriented Architecture (SOA)** where the **Node.js Backend** acts as the central orchestrator (gateway). This design ensures that the Client (Frontend) never communicates directly with sensitive components like the Database or the AI Service, thereby enforcing security and academic integrity rules centrally.

Although the system follows service-oriented principles through external services (Docker sandbox runner, Ollama), the backend is implemented as a modular monolithic application acting as a central gateway. This allows consistent policy enforcement (RBAC, exam constraints) and simplifies deployment while keeping integrations loosely coupled.

1.1. Architectural Flow:

1. **Frontend (React):** Serves as the user interface for students and instructors.
2. **Backend (Node.js/Express):** Manages authentication, business logic, and request routing.
3. **External Services:**
    a. **Docker Sandbox Runner:** Provides an isolated, per-request container for secure code execution.
    b. **AI Service (Ollama):** Manages LLM interactions for hint generation and question variations.
4. **Data Layer (PostgreSQL):** Handles persistent storage for user data (SQL) and flexible AI logs (JSONB).

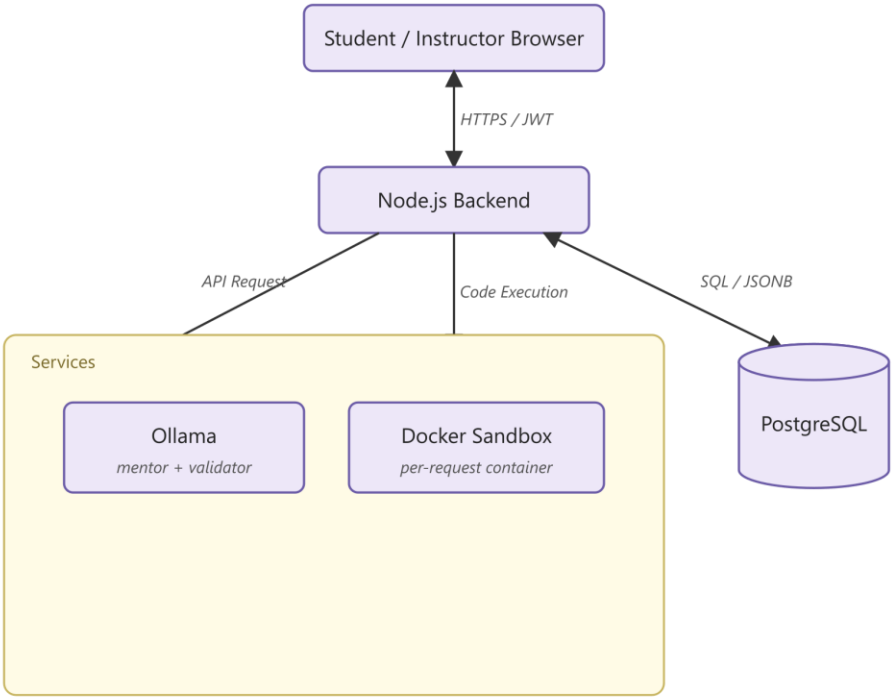

Figure 1: High-Level Architecture illustrating the central role of the Node.js backend in coordinating Frontend, Database, AI, and Sandbox services.

1.2. Operational Workflow

While the static architecture of the system is established on a centralized structure as illustrated in Figure 1, the dynamic process, ranging from a student's interaction with a problem to the instructor's retrieval of analytical data, requires operational synchronization between components. Figure 2 summarizes the end-to-end data flow and user workflows of the system.

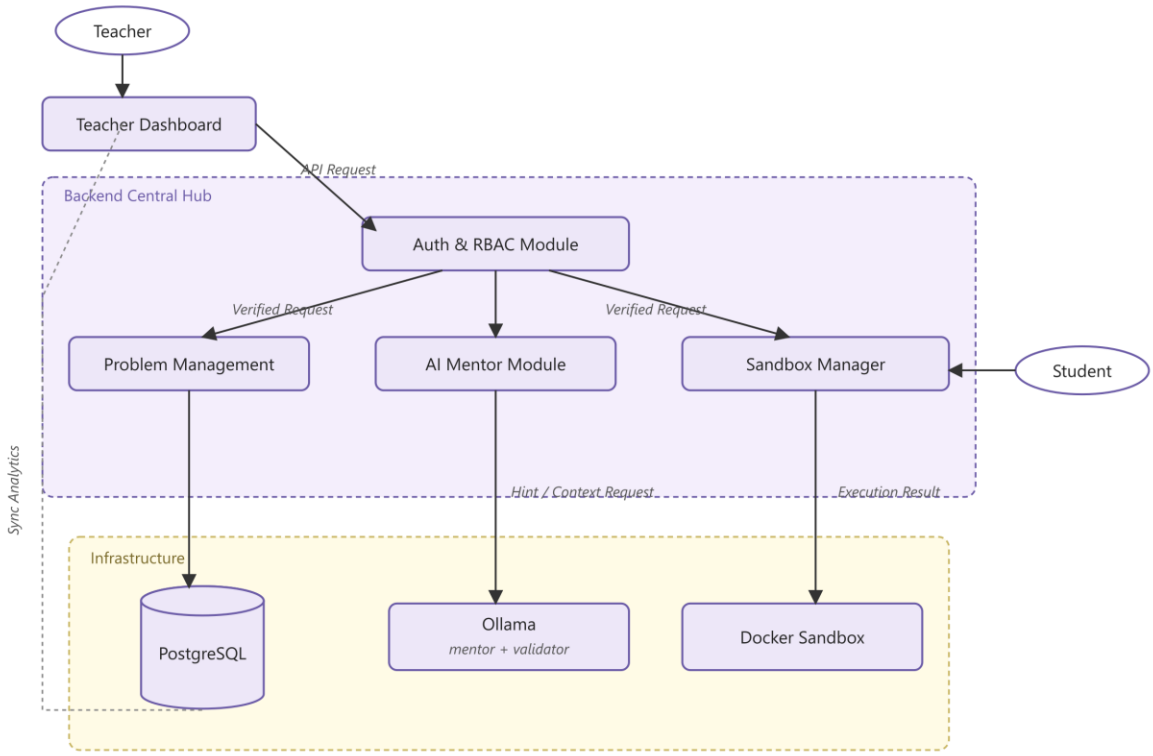

Figure 2: End-to-End Operational Flow. This diagram illustrates how instructor and student workflows are coordinated through the Backend Central Hub; specifically showing the concurrent operation of the AI Mentor module alongside code execution within the Sandbox environment.

2. System Modules

The backend logic is partitioned into five distinct modules to address specific functional requirements defined in the project scope.

2.1. Identity & Access Management (IAM) Module

Responsible for secure user entry and role management.

- **Authentication:** Implements JWT (JSON Web Tokens) for stateless authentication, handling secure login and session validation.
- **Access Control (RBAC):** Enforces permissions to distinguish between *Student* (limited access), *Teacher* (administrative access), and *Coordinator Teacher* (elevated academic management privileges) roles. The Coordinator Teacher can approve teacher registrations, view all students, and assign students to instructors, but is not a full system administrator.
- **Exam Security:** Manages Exam Mode controls, which disable AI features during live exams and enforce a controlled assessment window. Both backend API access and frontend assistive panels are restricted during the exam period to prevent academic dishonesty.

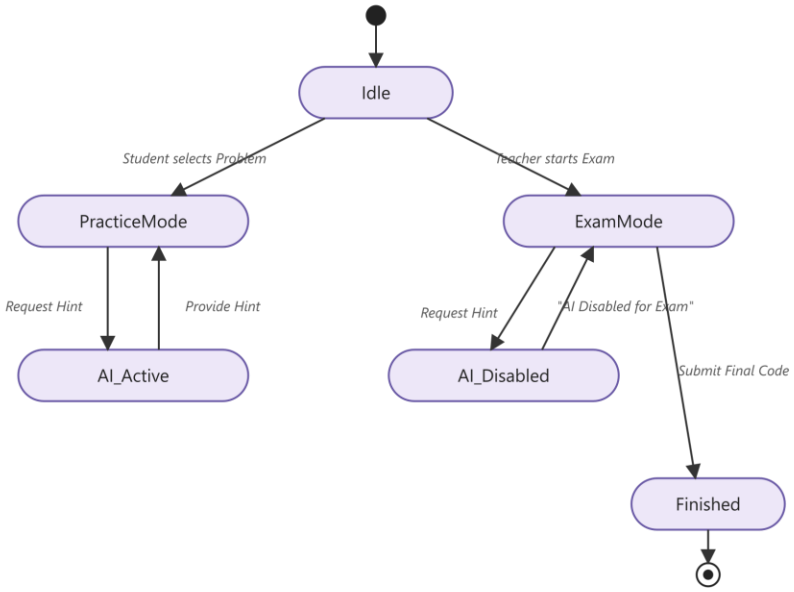

Figure 3: State Transition of Practice and Exam Modes. Status diagram showing the system's adoption status based on student and teacher actions, and the mode-based activity status of the AI mentoring service.

2.2. AI Mentor & NLP Engine Module

The core pedagogical engine that safeguards against solution leakage.

- **Hint Validation:** Implements a "Tutor/Student" simulation loop. A primary AI generates a hint and a secondary validation stage audits it to ensure the hint does not reveal the direct answer. Validation combines deterministic rule checks with an auxiliary AI verifier so that unsafe replies are caught before reaching the student.
- **Prompt Registry:** Manages a *Prompt Configuration Registry*, storing standardized templates to keep the AI in a strict 'mentor' role.
- **Conversation Memory:** Recent student–mentor exchanges are forwarded to the AI on each turn to support multi-turn debugging and concept follow-ups while keeping the prompt bounded.
- **Bilingual Support:** The mentor adapts its response language to the student's most recent message, allowing the same service to be used in both English and Turkish learning contexts.
- **Logging:** Records full *Interaction History* for every student-AI chat session to allow instructors to audit the guidance provided.

2.3. Code Execution & Sandbox Module

Manages the insecure nature of user-submitted code.

- **Isolation:** Relays code snippets and hidden test cases to a **Docker-based sandbox runner**, ensuring execution happens in a fresh, ephemeral container per request. Per-language container images guarantee a consistent and reproducible execution environment.
- **Result Parsing:** Processes raw execution outputs (stdout, stderr), memory usage, and execution time to provide feedback to the student.

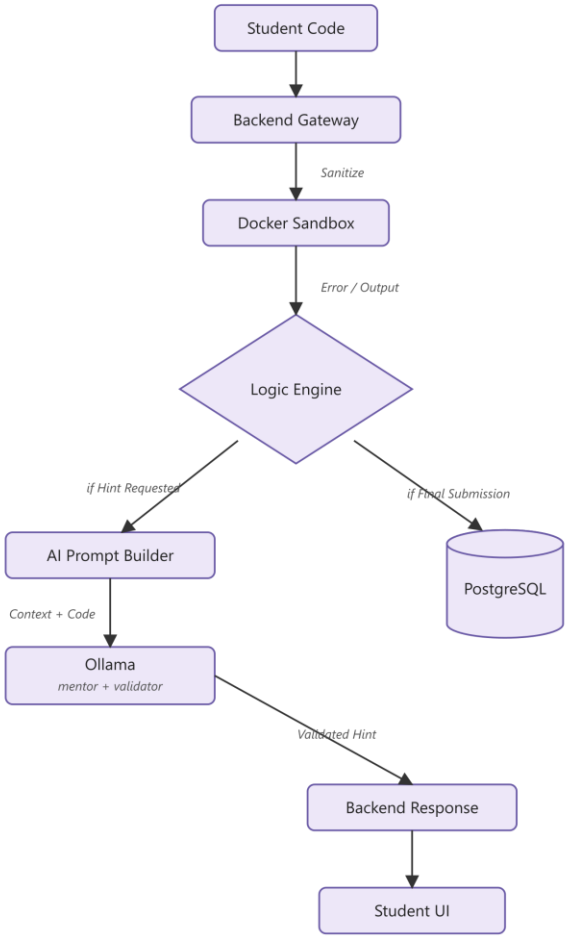

Figure 4: Data Flow Diagram illustrating the lifecycle of a student submission.

The process begins with code sanitization at the Backend Gateway, followed by execution in an isolated Docker sandbox. The Logic Engine then determines whether to route the resulting error/output to the AI Prompt Builder for pedagogical hint generation or to persistent storage in PostgreSQL for final evaluation.

2.4. Teacher Assistant Module

Automates content creation for instructors.

- **Structural Analysis:** Analyzes the logical structure (loops, conditions) of the instructor's reference code to capture its complexity profile.
- **Variation Generation:** Sends this structural context to the AI service to generate semantically similar problem variations or conceptual questions. All suggestions are reviewed and approved by the instructor before being added to the question bank.
- **Rubric Generation:** Suggests grading rubrics based on the complexity of the code structure.

2.5. Analytics & Reporting Module

Tracks student progress and system health.

- **Version Control:** Maintains *Versioned Code Snapshots* of student submissions, enabling a timeline view of how a solution evolved.
- **Performance Metrics:** Aggregates data on common error types and submission success rates to help instructors identify struggling students.
- **Submission Tracking:** Logs final submissions with precise timestamps to verify compliance with deadlines.

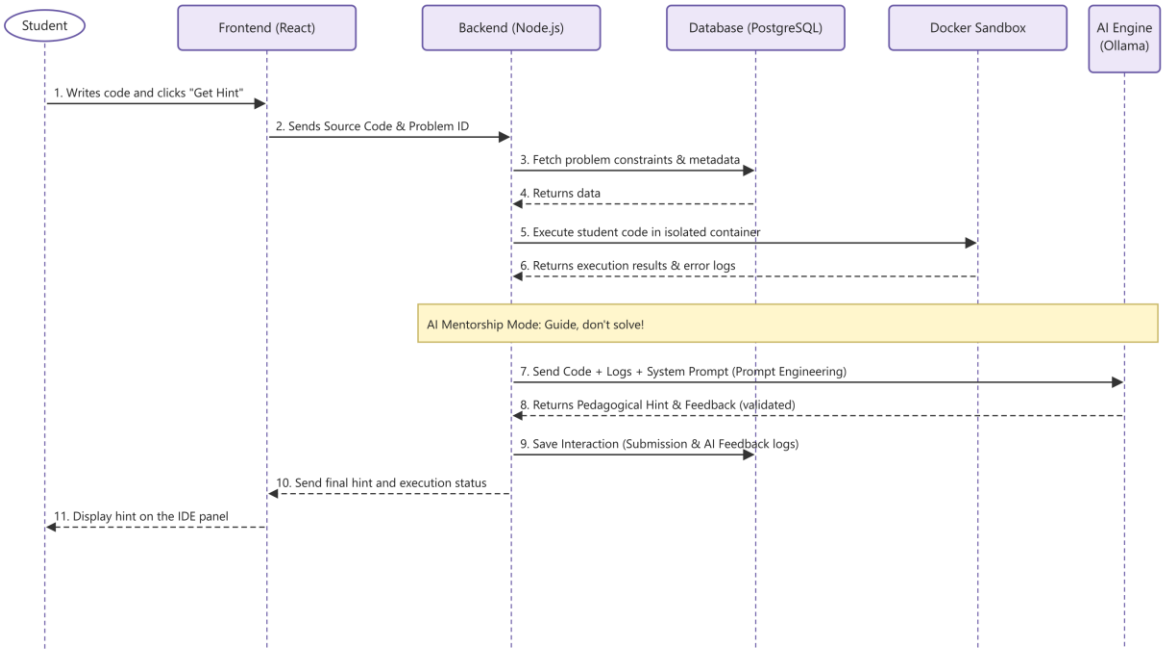

Figure 5: Sequence Diagram illustrating the request/response flow for AI hint generation.

3. API Design

The backend exposes a **RESTful API** to manage resources. The design prioritizes clear separation between student and instructor endpoints.

Core API Endpoints

Method	Endpoint	Description
POST	/api/auth/login	Authenticates credentials and returns a JWT.
GET	/api/problems	Retrieves a list of assigned problems and metadata for the user.
POST	/api/submit	Submits code and test cases to the Docker sandbox runner for execution.
POST	/api/ai/chat/stream	Requests a pedagogical hint; triggers internal validation logic and streams the validated reply.
POST	/api/ai/variation	Generates problem variations using structural analysis of the reference solution.
GET	/api/submissions	Fetches versioned code snapshots and interaction logs.
PATCH	/api/assignments/:id	Updates assignment settings, including exam-mode constraints that enable or disable AI help for a given assessment.

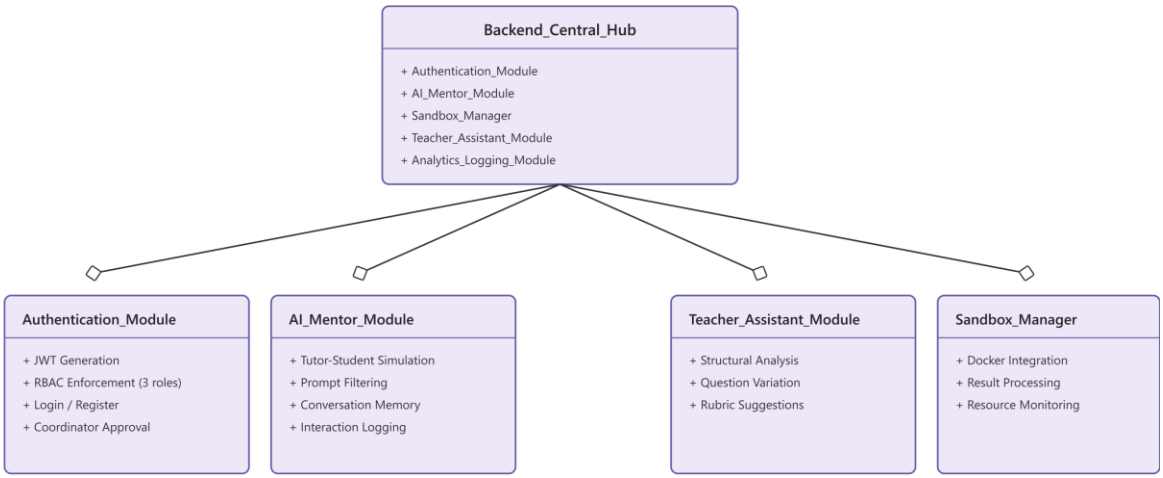

Figure 6: Backend Module Decomposition and Component Architecture.

4. Deployment & Infrastructure

The deployment strategy focuses on **system integrity**, **data privacy**, and **scalability** using containerization and hybrid storage solutions.

4.1. Infrastructure

- **Containerization (Docker):** The Backend, Database, AI service, and Frontend are deployed as containerized services. This ensures the system remains available and responsive under varying server loads by isolating dependencies.
- **Sandbox Runner:** The execution sandbox runs as an isolated, per-request Docker container so that user-submitted code cannot affect the main backend server or persist across submissions.

4.2. Database Design

A hybrid database approach is used to balance integrity with flexibility:

- **PostgreSQL (Relational):** Used for *Identity & Access Management* and *Problem Assets*. It ensures *ACID compliance*, which is critical for maintaining accurate academic records and grades.
- **JSONB (Flexible):** Implemented within PostgreSQL to store *AI Prompts*, *Interaction Logs*, and *Dynamic Rubrics*. This allows the system to adapt to the unstructured nature of LLM outputs without altering the database schema.

The database layer is designed to support secure user management, controlled assignment workflows, and auditability of student submissions and AI-assisted interactions. Role-based access control (RBAC) ensures a clear separation between student, instructor, and coordinator privileges.

Figure 7 presents the Entity-Relationship (ER) diagram of the core data model. Users, roles, problems, assignments, enrollments, and submissions are modeled to support versioned code snapshots and detailed submission tracking. AI-assisted interactions, grading rubrics, and feedback artifacts are persistently logged to enable instructor review and to preserve academic integrity.

This data model directly supports the system architecture by enabling secure access control, transparent evaluation, and controlled AI mediation.

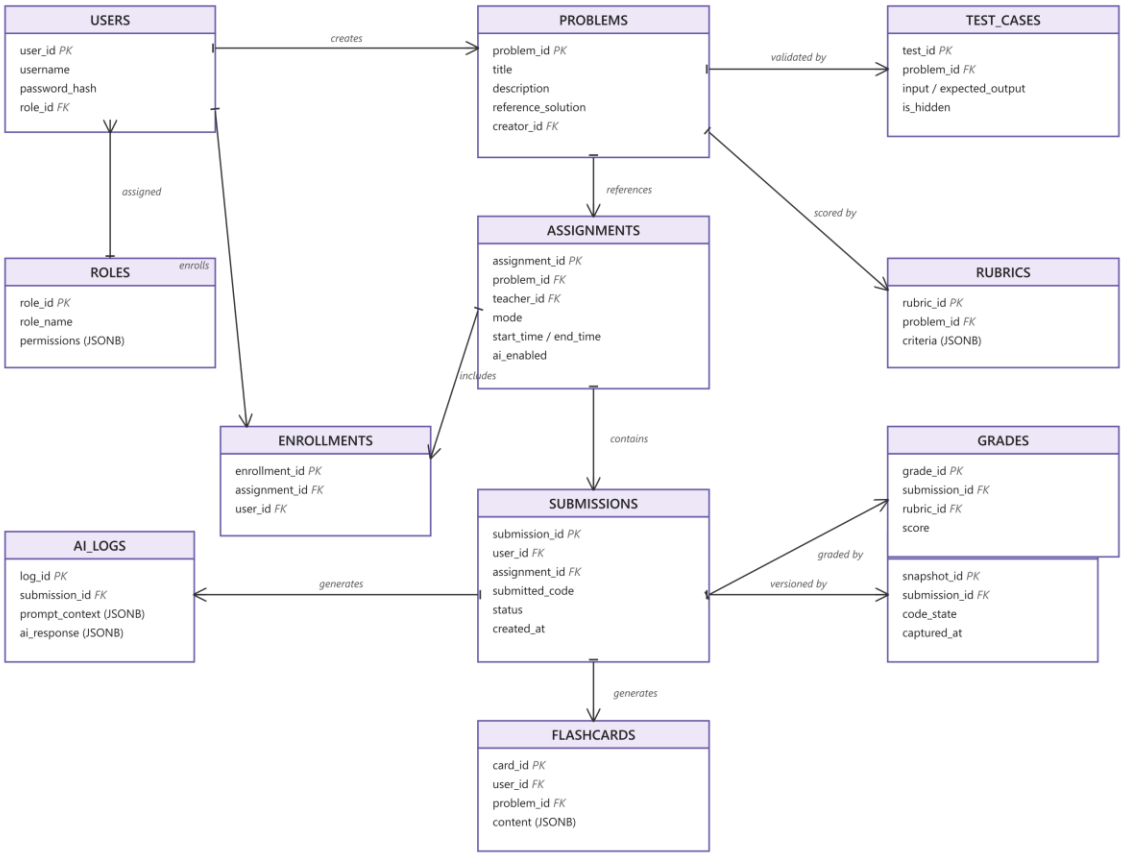

Figure 7: Entity-Relationship Diagram of the Core Data Model.

4.3. AI Model Deployment

- **Interface Layer:** The backend communicates with the AI service through Ollama's HTTP API, sending fully composed prompts and consuming the generated responses directly.
- **Hybrid Support:** The architecture supports both **Cloud-based LLMs** and **On-Premise (Local) LLMs** (e.g., via Ollama). This allows institutions to deploy the system locally to ensure that sensitive student data never leaves the campus network.

5. Non-Functional Requirements

Beyond functional requirements, the AI-Assisted Programming Platform is designed to satisfy key non-functional requirements that ensure reliability, scalability, security, and maintainability in academic environments, particularly during high-concurrency scenarios such as laboratory sessions and live examinations.

Scalability: The system supports multiple concurrent users, especially during exams, through containerized deployment that allows horizontal scaling of backend and sandbox services without architectural changes.

Performance: Performance-critical operations, including AI inference and code execution, are isolated from core backend logic. Non-blocking I/O and asynchronous request handling minimize latency, while reusable prompt templates reduce processing overhead.

Availability: Core functionalities such as problem access, code submission, and result storage remain operational even if auxiliary services like the AI inference engine become temporarily unavailable, ensuring uninterrupted exam execution.

Security and Academic Integrity: All sensitive operations are mediated by the backend central hub, enforcing role-based access control, exam mode constraints, and comprehensive audit logging to prevent misuse and maintain fairness.

Maintainability: A modular backend architecture enables independent evolution of system components, simplifying debugging, feature extension, and long-term maintenance.

Observability: Structured logs and interaction histories, particularly for AI-assisted sessions, support instructor auditing, troubleshooting, and post-exam analysis.

5.1 Failure Scenarios & Mitigation Strategies

The system is designed to handle partial failures gracefully, ensuring controlled degradation and recovery without compromising academic integrity.

AI Service Unavailability: If the AI inference service becomes unreachable, AI-assisted features are disabled while the rest of the platform remains functional, and users are informed accordingly.

Hint Validation Failure: AI-generated hints that fail internal validation checks are blocked or replaced with generic conceptual guidance to prevent solution leakage.

Sandbox Execution Timeout or Resource Exhaustion: User code exceeding execution time or memory limits is safely terminated, and a controlled error response is returned without exposing internal details.

Database Connectivity Issues: Transient database failures are handled through retry mechanisms or fallback read-only behavior, preserving data consistency and submission integrity.

Authentication or Authorization Errors: Invalid or expired authentication tokens result in immediate request rejection and enforced re-authentication.

Abuse Prevention During Exams: Rate limiting and request throttling are applied to AI-related endpoints during exam mode to prevent excessive hint requests and resource misuse.

6. Design Decisions & Rationale

- **Why Node.js?** Chosen for its non-blocking I/O capabilities, which are essential for handling concurrent connections to the AI Service, Sandbox, and Database without latency.

- Why a Docker-based Sandbox? Decoupling code execution from the main backend via an ephemeral per-request container is a critical security decision to prevent user-submitted code from accessing file systems or environment variables on the main server. Per-language container images additionally ensure a reproducible execution environment across submissions.
- Why Structural Analysis for Variations? Plain text manipulation is insufficient for generating logically consistent problem variations. Capturing the structural complexity of the reference solution (loops, conditions, variable scope) allows the AI to produce variations that preserve the intended difficulty and concept coverage rather than just paraphrasing the problem description.

Implementation Details

1. Introduction

This document explains the implementation details of the AI-Assisted Programming Platform for Education and Assessment. The platform combines a browser-based programming environment, a Docker-isolated code execution sandbox, a two-stage AI Mentor (a 30-billion-parameter mentor model audited by a 3-billion-parameter validator model), automated grading, rubric-based teacher review, analytics, and a structured tutorial library.

The platform provides separate interfaces for students and teachers. Students can access programming assignments, solve problems in a browser-based coding environment, run and submit code, receive AI mentor support, review tutorials, track their progress, and study feedback flashcards. Teachers can create and manage assignments, enroll students, manage the question bank, review submissions, generate rubrics, grade student work, and analyze student performance.

In addition to the standard teacher role, the system includes a higher-privileged teacher role referred to as the **Coordinator Teacher**. The Coordinator Teacher is not a full system administrator, but has additional academic management privileges such as approving teacher registrations, viewing all students, and assigning students to other teachers.

A key purpose of the system is to support AI-assisted learning while also protecting academic integrity. For this reason, the platform includes an **Exam Mode** lockdown mechanism that disables AI Mentor access, hides assistive panels, and gates submissions to a teacher-defined time window during exam-style assignments.

2. System Overview

The system is a full-stack web application deployed as a set of Docker Compose services. Users access the platform through a browser and select the appropriate portal according to their role. The system separates the learning and assessment workflows into role-based modules.

The main platform capabilities are:

- Role-based student and teacher portals
- JWT-based authentication and authorization
- Programming assignment management
- Student enrollment by group or individually
- Question bank management
- Multi-file, multi-language code editor (Monaco)
- Docker-isolated code execution and automated testing
- Interactive WebSocket terminal with clickable error-line links
- Submission history tracking
- Context-aware AI Mentor support with two-stage validation

- Teacher-controlled Exam Mode lockdown
- Student analytics and learning progress tracking
- Feedback flashcards generated from mistakes and feedback
- Rubric-based grading
- AI-assisted rubric generation
- AI-assisted grading suggestions
- AI-assisted question variation generation
- Block-based tutorial library (~97 C tutorials in 8 categories / 23 sub-categories) with per-section PDF export
- Class-level analytics
- Teacher registration approval by the Coordinator Teacher
- Student-to-teacher assignment management by the Coordinator Teacher

3. Technology Stack

The platform is implemented with a modern full-stack JavaScript/TypeScript architecture. Concrete framework and runtime choices are listed below.

3.1 Frontend

The frontend is a single-page application built with **React 18** (Vite-bundled), **Material UI 7** for components and theming, **react-router 7** for routing, the **Monaco editor** for the code editor (the same engine used by VS Code), **xterm.js** with the fit addon for the interactive terminal, **recharts** for analytics charts, **marked** + **DOMPurify** for safe markdown rendering, and **jsPDF** for client-side PDF export of tutorial sections.

The student coding workspace includes an IDE-like editor that supports multiple files per problem (EditorTabBar) and displays language-specific source files such as main.c, main.py, main.js, Main.java, Program.cs, or main.cpp.

3.2 Backend

The backend is a **Node.js + TypeScript** service running on the **Express** framework. Database access is handled through the **Prisma ORM** against a PostgreSQL database. Mentor streaming responses are delivered over Server-Sent Events at /api/ai/chat/stream. The backend is responsible for authentication, authorization, assignment operations, problem management, submission handling, code execution requests, AI Mentor communication, analytics, grading, and privileged teacher operations.

3.3 Database

The system uses **PostgreSQL** for persistent storage. The schema is defined and versioned in prisma/schema.prisma with fifteen committed migrations applied automatically at container start. The database stores users, roles, teacher-student relationships, student groups, problems, test cases, assignments, enrollments, submissions, AI interactions, rubrics, grades, feedback flashcards, and system-level settings such as exam mode.

3.4 Code Execution Layer

The code execution layer is a **Docker-based runner** implemented in backend/src/services/dockerRunner.ts. Each Run/Submit spawns an ephemeral container (--rm) for the appropriate language image and pipes the student's source through stdin/stdout. This layer replaced an earlier Judge0/isolate-based runner that was incompatible with cgroup v2 on the host. The full design is described in Section 16.

3.5 AI Integration

AI inference is provided by **Ollama** serving two open-weights models locally on an NVIDIA L4 GPU (23 GB VRAM):

- qwen3-coder:30b (~20 GB VRAM) - the mentor model. Responsible for hints, explanations, and Socratic guidance.
- qwen2.5:3b-instruct (~2.4 GB VRAM) - the validator model. Audits each mentor reply against the academic-integrity policy.

Both models are pinned in VRAM with OLLAMA_KEEP_ALIVE=-1. The full two-stage architecture is described in Section 17.

Summary of the technology stack:

Layer	Technology	Purpose
Frontend framework	React 18 + Vite	Single-page application bundle
UI components	Material UI 7	Design system and theming
Editor	Monaco editor	Multi-language code editor
Terminal	xterm.js + fit addon	Interactive program I/O
Charts	recharts	Analytics visualisations
Markdown	marked + DOMPurify	Safe markdown rendering
PDF export	jsPDF	Tutorial section export
Backend runtime	Node.js + TypeScript	API service
Backend framework	Express	HTTP routing
ORM	Prisma	Database access and migrations

Layer	Technology	Purpose
Database	PostgreSQL	Persistent storage
Sandbox	Docker (Compose)	Code execution isolation
LLM runtime	Ollama	Local model serving
Mentor model	qwen3-coder:30b	Socratic hint generation
Validator model	qwen2.5:3b-instruct	Policy audit of mentor replies
Auth	JWT bearer tokens	Session management
Deployment	Docker Compose on GCE VM (L4 GPU)	Orchestration

4. User Roles and Permissions

The platform uses role-based access control. Each user can only access the pages and actions allowed for their role.

4.1 Student

Students can:

- View assigned homework, practice tasks, and exams
- Open programming problems
- Write code in the online editor (multi-file)
- Run code and view compiler/runtime output in an interactive terminal
- Submit solutions for automated evaluation
- View their submission history
- Ask the AI Mentor for help during normal learning activities
- Use the Debug-Prints shortcut to ask the mentor for instrumentation
- Access tutorials and export sections as PDF
- View personal analytics
- Review feedback flashcards

Students cannot manage assignments, view other students' submissions, edit question bank content, approve teachers, or access teacher management pages.

4.2 Standard Teacher

Standard teachers can:

- Create and manage assignments for their students

- Enroll students or groups into assignments
- Create and edit programming problems
- Manage test cases (public and hidden) and starter code
- Use AI-assisted problem generation tools
- Create or generate grading rubrics
- Review and grade submissions
- View analytics for their assigned students
- Enable Exam Mode for relevant assessment scenarios

4.3 Coordinator Teacher

The Coordinator Teacher is a higher-privileged teacher role designed for a department coordinator or course coordinator. The Coordinator Teacher is not a full administrator. The Coordinator Teacher can:

- Access all standard teacher features
- Approve or reject teacher registration requests
- View all students in the system
- Assign students to other teachers
- Manage teacher-student distribution
- Monitor broader course-level or department-level teaching activity

5. Authentication and Authorization

Authentication is implemented with **JWT bearer tokens**. The platform starts with a portal selection screen where users choose either the Student Portal or the Teacher Portal. Each portal provides a role-specific sign-in flow.

After successful login, the backend issues a signed JWT containing the user's role. The frontend stores the token in browser storage and sends it as an Authorization: Bearer <token> header on every API request. Each protected route runs an Express middleware that decodes the token, validates the role, and rejects the request if the user is not permitted to perform the action.

Student navigation includes: Dashboard, Assignments, Analytics, Flashcards. Teacher navigation includes: Dashboard, Assignments, Students, Question Bank, Grading, Analytics, and (for Coordinator Teacher only) Approvals.

6. Student Portal Implementation

6.1 Student Dashboard and Navigation

The Student Portal provides a learning-focused interface. Students can move between assignments, analytics, flashcards, and AI Mentor-supported programming activities. The navigation is designed to keep the main learning features easily accessible.

6.2 Assignment Listing

Students can view programming tasks assigned to them. Assignments are categorized as homework, practice, or exams. Each assignment exposes title, topic tags, difficulty level, supported programming language, deadline, and completion status. The list also surfaces aggregate progress (how many of the assigned tasks have been completed).

6.3 Problem-Solving Workspace

The problem-solving workspace is one of the core student features. It is divided into three main areas:

1. Assignment and tutorial panel: switches between available problems and the block-based tutorial library.
2. Problem and editor panel: shows the problem description, examples, language selection, source file tabs, and the Monaco editor.
3. AI Mentor panel: contextual chat with the two-stage mentor pipeline (collapses to single-column in Exam Mode).

The center panel includes: problem title, difficulty level, programming language, problem statement, example input/output, code editor with multi-file tabs (EditorTabBar), Run button, Submit button, an interactive xterm.js terminal area, and submission history.

The editor supports per-language starter files. A C problem opens with a main.c template, a Python problem with main.py, a Java problem with Main.java, C# with Program.cs, JavaScript with main.js, and C++ with main.cpp. Students may add, rename, or close additional source files within the same problem.

6.4 Run Workflow

The Run workflow allows students to test their code interactively before submitting it for grading.

4. The student writes or edits code in the browser.
5. The student clicks Run.
6. The frontend opens a WebSocket session for the terminal and POSTs the source code, language, and active file list to the backend.
7. The backend spawns a Docker container for the selected language image via dockerRunner.ts.
8. Compilation (if needed) runs first, with a 30-second timeout.
9. On success, the binary runs interactively; stdin/stdout/stderr are bridged through the WebSocket to xterm.js.
10. Compiler messages and stack traces are scanned for line numbers; matches become clickable links that jump the editor to the offending line.

6.5 Submit Workflow

The Submit workflow evaluates the student's solution against the assignment's test cases.

11. The student submits the current code.
12. The backend retrieves all test cases for the problem (public and hidden).
13. Using the compile-once-run-N session model, dockerRunner.ts compiles the code once and runs the resulting binary against each test case in sequence.
14. For every test case the runner records stdout, stderr, exit code, and wall-clock execution time.
15. The platform compares actual output with expected output for each test case.
16. Per-test results plus the overall status (accepted / failed / compile_error / runtime_error / time_limit) are stored in the submission record.
17. The student sees per-test results and a refreshed submission history.

Submission history rows store status, programming language, total execution time, timestamp, related assignment/problem, and the full source code at submission time. This historical record supports analytics, grading, and flashcard generation.

6.6 Compiler and Runtime Feedback

Compiler and runtime feedback is rendered directly inside the student interface. The terminal uses **xterm.js** with a custom link provider that turns patterns like line 14, main.c:23, or stack-trace file references into clickable links. Clicking a link moves the editor cursor to the referenced line.

If a student writes an assignment operator where a comparison was intended, the compiler error appears in the terminal panel and the student can either fix it directly or ask the AI Mentor for an explanation through the chat panel.

6.7 AI Mentor

The AI Mentor is integrated into the problem-solving workspace. It receives contextual information about the current problem, the student's code, recent run output, the last 20 conversation turns, and (when available) the editor cursor context. The mentor can:

- Provide hints calibrated to the current hint level (1 = nudge, 5 = explicit walkthrough without code)
- Explain programming concepts
- Ask Socratic guiding questions
- Explain compiler and runtime errors
- Review parts of the student's code without rewriting it
- Reply in English or Turkish (detected via mentor.ts:detectMessageMode)
- Insert temporary debug prints around the cursor via the Debug-Prints button

Every mentor reply passes through a second-stage validator before reaching the student. The full architecture is described in Section 17.

6.8 Exam Mode Behaviour for Students

Exam Mode is a full lockdown of the student workspace, not just a flag on the AI panel. When Exam Mode is active, the frontend (StudentWorkspace.jsx + AppHeader.jsx):

- Removes the AI Mentor panel and chat input from the DOM
- Hides the assignments panel and the submissions history sidebar
- Collapses the workspace into a single-column layout
- Hides administrative navigation items and the notifications bell in AppHeader (lockdown prop)
- Installs a beforeunload guard that warns on accidental tab/window close
- Installs a contextmenu guard that disables right-click
- Gates submissions to the teacher-defined start/end time window
- Shows the 'Finish Exam' button only after every visible test passes

On the backend, the AI Mentor route checks exam state on every request and rejects mentor calls during an active exam window. This dual enforcement (frontend hides, backend blocks) prevents bypass via direct API calls.

6.9 Tutorials

The Tutorials module ships with approximately **97 C tutorials** organised under 8 main categories (C Tutorial, C Functions, C Files, C Structures, C Enums, C Memory, C Errors, C More) and 23 sub-categories. Tutorials are stored as JSON files under backend/src/data/tutorials/c/ and served via a 3-level GET /api/tutorials/index endpoint that returns category - subCategories - topics.

Each tutorial follows a **block-based schema** that mirrors w3schools' interleaved layout. The block types are:

Block type	Renders as
text	Markdown-rendered paragraph
code	Monaco read-only editor with syntax highlighting
syntax	Monaco editor with a 'SYNTAX' label
note	Yellow Alert callout (InfoOutlined icon)
goodtoknow	Blue Alert callout (LightbulbOutlined icon)
list	Bulleted list with inline markdown items

The TutorialModal component renders both the new block-based schema and the legacy body/code schema so older tutorials continue to work. Students can export any tutorial section as a PDF (jsPDF) directly from the side panel.

6.10 Student Analytics

The Student Analytics page gives students a personal overview of their learning progress. Metrics include total submissions, success rate, number of solved problems, number of AI hints used, learning trend, activity history, error breakdown, and progress over time.

6.11 Feedback Flashcards

The Flashcards module converts previous feedback, code issues, and learning suggestions into reusable study cards. Cards are generated by an LLM pass with the following constraints (flashcardService.ts):

- format: "json" mode, temperature 0.1, num_predict 3000
- Target card count tuned to difficulty: 2 / 3 / 5 / 7 cards
- Base timeout 180s, +30s per attempt, capped at 8 minutes
- Post-filters: isValidCard() (schema check), dedupeCards() (Jaccard similarity), hasGrounding() (every card must quote a specific token from the student's code)
- Prompt includes explicit GOOD/BAD examples to anchor card quality

Each card contains feedback type, related problem, explanation of the issue, the problematic code snippet, a suggested improvement, and a before/after code comparison. Examples include redundant code, inefficient algorithms, incorrect output formatting, missing edge case handling, and cleaner code suggestions.

7. Teacher Portal Implementation

7.1 Teacher Dashboard

The Teacher Dashboard provides a high-level overview of teaching activity: total students, total attempts, accepted submissions, AI hints used, assignment summaries, due dates, assignment status, and assignment modes.

7.2 Exam Mode Control

Teachers activate Exam Mode per assignment. The setting is stored on the assignment row and consulted by both the frontend lockdown logic (Section 6.8) and the backend AI Mentor route. Exam Mode can be applied to all enrolled students, specific student groups, or selected assessment scenarios. Frontend lockdown plus backend AI-call blocking together preserve academic integrity.

7.3 Assignment Management

The Assignments page lets teachers create, edit, publish, and manage assignments. The assignment form is restructured so the problem selector is the first field; selecting a problem auto-fills the title, description, and language from the linked problem (AssignmentModal.jsx). The form captures: title, linked problem, description, due date, enrolled students, status, mode, visibility rules, and (when applicable) exam start and end times.

Enrollment is integrated into the assignment form via the EnrollModal component running in 'select' mode (no API calls until save), so a teacher can build an assignment and its roster in a single dialog.

7.4 Assignment Modes

The system supports two assignment modes: homework/practice and exam. In homework or practice mode, students can access problems and use AI Mentor support. In exam mode, the teacher defines a controlled assessment window: students are restricted by exam start and end times, AI Mentor access is disabled, and the frontend enters lockdown.

7.5 Student Enrollment

Teachers enroll students into assignments using groups, individual selection, or a combination of both. The flow supports selecting complete student groups, selecting individual students, combining group and individual selection, viewing selected student count, and saving enrollment changes atomically.

8. Coordinator Teacher Features

8.1 Teacher Registration Approval

When a new teacher registers, the Coordinator Teacher reviews the request from the Approvals page and approves or rejects it. This prevents unauthorized teacher accounts from gaining access to teaching features.

8.2 Global Student Visibility

Unlike a standard teacher (who sees only assigned students), the Coordinator Teacher can list every student in the system.

8.3 Assigning Students to Teachers

The Coordinator Teacher can assign students to standard teachers, supporting workflows where a department coordinator distributes students across instructors or teaching assistants.

8.4 Difference Between Coordinator Teacher and Administrator

The Coordinator Teacher is not a full administrator. The role is focused on academic coordination rather than system administration. It cannot edit user identities, alter system flags directly, or manage infrastructure - those would require a true admin role.

9. Question Bank Implementation

9.1 Question Bank Management

The Question Bank stores reusable programming problems. Teachers can search, filter, create, edit, and manage questions. Each question can include: title, problem description, difficulty level, topic tags, supported languages, starter code, hidden reference solution, public test cases, hidden test cases, rubric information, and attempt statistics.

9.2 Problem Creation

Teachers create new programming questions using a structured form covering metadata, description, starter code, reference solution, and test cases. The hidden reference solution is used for teacher-side validation and is never sent to students. Test cases can be configured as visible or hidden. Public test cases help students understand expected behaviour; hidden test cases improve assessment reliability.

9.3 AI-Assisted Question Generation

Teachers can request the LLM to generate easier, similar, or harder versions of an existing problem. The generated question appears in a preview pane for the teacher to review, edit, reject, or approve before it is added to the question bank. This keeps the teacher in control while reducing the time required to create new assessment material.

10. Rubric Implementation

Rubrics allow teachers to grade submissions in a structured and consistent way. A rubric can include multiple criteria such as correctness, algorithm efficiency, edge case handling, code readability, use of appropriate programming constructs, and output formatting. Each criterion has a name, description, maximum points, and a scoring guide.

The platform also supports AI-assisted rubric generation. Teachers can generate a rubric suggestion for a selected problem and then approve or modify it before use.

11. Student Management and Progress Tracking

The Students page allows teachers to view student information and learning progress. Student records include name, email, academic year, assigned teacher, group membership, completed problems, and progress percentage.

Teachers can search and filter students and open individual student details to review deeper performance information: attempts, hints used, solved languages, difficulty distribution, language performance, problem breakdown, and top errors.

Standard teachers see only their assigned students; the Coordinator Teacher sees all students and can assign students to teachers.

12. Grading Implementation

The Grading page allows teachers to evaluate student submissions through this workflow:

18. Teacher selects an assignment.
19. The system lists enrolled students and their submission statuses.
20. Teacher opens a student's submission.
21. The system displays code, per-test results, runtime, and overall status.
22. Teacher scores the submission using rubric criteria.
23. Teacher optionally writes comments.
24. Teacher saves the final grade.

The grading screen distinguishes public and hidden test results so teachers can see whether a solution passes only the visible examples or also the hidden evaluation cases.

12.1 AI-Assisted Grading Suggestions

The AI Suggest feature proposes rubric scores and comments based on the submitted code, test results, rubric criteria, output correctness, and code-quality indicators. The teacher remains responsible for the final grade - AI suggestions are decision support, not automatic grading.

12.2 Exporting Grades

The grading module exports grading data as an Excel file using the SheetJS (xlsx) library. This helps teachers keep offline records or submit grades to external systems.

13. Class Analytics Implementation

The Class Analytics page provides class-level performance information rendered with recharts: number of students, total submissions, accepted submissions, class accept rate, number of problems, weekly submission activity, hardest problems, accept rate by problem, and common error types. For the Coordinator Teacher, analytics can also support broader monitoring across multiple groups or teachers.

14. Database Design

The database schema is defined in prisma/schema.prisma and migrated automatically at container start (15 migrations applied non-interactively by prisma migrate deploy). Main logical entities:

- User
- Role
- Student profile
- Teacher profile
- Teacher registration request
- Teacher-student assignment
- Student group
- Problem
- Test case (public/hidden flag)
- Assignment
- Assignment enrollment
- Submission
- Per-test execution result
- AI interaction log
- Feedback flashcard
- Rubric
- Rubric criterion
- Grade

- Exam mode setting
- System flag

14.1 User and Role Data

The user model stores authentication-related information (hashed password, email, display name) and role assignment. Roles define whether a user is a student, standard teacher, or Coordinator Teacher.

14.2 Problem and Test Case Data

Problems store the problem statement, difficulty, topic tags, supported languages array, starter code per language, and the hidden reference solution. Test cases are linked to problems and carry a public/hidden flag plus stdin/expected-stdout.

14.3 Assignment and Enrollment Data

Assignments link problems to students or groups. The enrollment row determines which students can access a given assignment and tracks per-student completion status.

14.4 Submission Data

Each submission stores the student, assignment/problem, language, submitted code, overall status, total execution time, per-test results, and timestamp.

14.5 AI Data

AI interactions store the student question, the mentor response, hint level, validator verdict, validator violation list, generated flashcards, and a timestamp. This table powers analytics, hint counting, and flashcard generation.

14.6 Grading Data

Grades are linked to submissions and rubric criteria, allowing per-criterion scoring and structured teacher feedback.

15. Backend Workflow Details

15.1 Authentication Workflow

25. User submits login credentials.
26. Backend validates the credentials.
27. Backend identifies the user role and issues a signed JWT.
28. JWT is returned to the frontend and stored client-side.
29. Frontend renders the correct portal and navigation based on the JWT role claim.
30. Every subsequent API request carries the JWT in the Authorization header; backend middleware verifies it.

15.2 Assignment Workflow

31. Teacher creates or edits an assignment via AssignmentModal.
32. Teacher selects a problem from the question bank (title/description/language auto-populate).

33. Teacher defines assignment metadata and mode.
34. Teacher enrolls students or groups using EnrollModal in 'select' mode.
35. On save, the backend stores the assignment and writes all enrollment records in a single transaction.
36. Students see only the assignments available to them.

15.3 Code Execution Workflow

37. Student writes code in the editor.
38. Student clicks Run or Submit.
39. Frontend sends source code, language, and active file list to the backend; for Run it also opens a WebSocket for the terminal.
40. Backend validates access to the assignment.
41. Backend invokes dockerRunner.ts, which spawns the language-appropriate Docker container with --rm.
42. Container compiles or interprets the code.
43. For Submit, the runner reuses the compiled binary across all test cases (compile-once-run-N session).
44. Per-test stdout/stderr/exit-code/wall-time are collected.
45. Backend stores submission and per-test results.
46. Frontend displays output, errors, test status, and execution time.

15.4 AI Mentor Workflow

47. Student sends a message to /api/ai/chat/stream (Server-Sent Events).
48. Backend checks whether AI access is allowed (Exam Mode would block here).
49. Backend builds the mentor prompt from: assignment text, current student code, recent run status + stdout/stderr, last 20 chat turns, hint level, locale, and (optionally) [FOCUSED CODE NEAR CURSOR] from the editor.
50. Mentor model (qwen3-coder:30b) streams a draft reply.
51. Validator pipeline runs the draft through heuristic checks (validator.ts) and, if needed, the small validator model (qwen2.5:3b-instruct).
52. If the reply is rejected, the backend either repairs it or returns a safe fallback hint.
53. Approved reply is streamed to the student; the interaction is stored for analytics and flashcard generation.

15.5 Teacher AI Workflow

54. Teacher requests AI support for problem generation, rubric generation, or grading suggestion.
55. Backend collects the relevant problem, submission, or rubric context.
56. AI generates a suggestion (JSON-mode for rubrics/cards).
57. Teacher reviews the suggestion.
58. Teacher approves, edits, or rejects the suggestion - nothing is saved without explicit approval.

15.6 Coordinator Teacher Workflow

59. Coordinator Teacher reviews teacher registration requests.
60. Coordinator Teacher approves or rejects each request.
61. Coordinator Teacher views all students.
62. Coordinator Teacher assigns students to appropriate teachers.
63. Standard teachers then manage their assigned students.

15.7 API Surface (selected)

The backend exposes a REST API plus one streaming endpoint. Selected routes:

Method	Path	Purpose
POST	/api/auth/login	Issue JWT
POST	/api/auth/register	Student/teacher registration
GET	/api/assignments	List assignments for the current user
POST	/api/assignments	Create assignment (teacher)
POST	/api/assignments/:id/enroll	Enroll students/groups
GET	/api/problems/:id	Fetch problem detail
POST	/api/run	Run code (interactive, WebSocket)
POST	/api/submit	Run all test cases, store submission
GET	/api/submissions	Submission history
POST	/api/ai/chat/stream	Mentor chat (SSE stream)
POST	/api/ai/flashcards	Generate feedback flashcards
POST	/api/ai/rubric	Generate rubric suggestion
POST	/api/ai/grade	Generate grading suggestion
POST	/api/ai/variation	Generate problem variation

Method	Path	Purpose
GET	/api/tutorials/index	3-level tutorial TOC
GET	/api/tutorials/:tag/:lang	Tutorial content (blocks)
GET	/api/analytics/student/:id	Per-student analytics
GET	/api/analytics/class	Class-level analytics
POST	/api/grades	Save rubric grade
GET	/api/grades/export	Excel export
GET	/api/coordinator/approvals	Pending teacher registrations
POST	/api/coordinator/approvals/:id	Approve/reject

16. Code Execution Sandbox

Code execution is implemented as a Docker-based runner in backend/src/services/dockerRunner.ts. This section describes the design and the migration that led to it.

16.1 Why Docker (not Judge0 / isolate)

An earlier iteration of the platform used the open-source **Judge0** backend, which relies on **isolate** for sandboxing. isolate v1.x is built on cgroup v1, which is no longer the default on modern Linux hosts. The host VM ships with cgroup v2 only, and isolate v1 fails to create sandboxes in that environment - producing time-limit-exceeded errors on every submission regardless of the program. Rewriting isolate to support cgroup v2 was out of scope, so the runner was rebuilt around Docker itself.

16.2 Architecture

Each Run/Submit call ends in runInDocker() or createDockerSession() (the compile-once-run-N variant used by Submit). Both spawn an ephemeral container with the --rm flag, mount the student's source files into /code, pipe stdin/stdout/stderr through Node streams, and tear down the container when the process exits. The backend has a stdin error handler in place to absorb EPIPE failures cleanly.

Per-language images and compile commands:

Language	Image	Compile / run command
Python	python:3.12-slim	python3 main.py
JavaScript	node:20-slim	node main.js
C	gcc:13	gcc -O2 -o /code/bin /code/main.c && /code/bin
C++	gcc:13	g++ -O2 -o /code/bin /code/main.cpp && /code/bin
Java	eclipse-temurin:21-jdk-alpine	javac Main.java && java Main
C#	mcr.microsoft.com/dotnet/sdk:8.0-alpine	dotnet build --nologo -o /code/bin /code/app.csproj && dotnet /code/bin/app.dll

16.3 Timeouts and Resource Limits

- COMPILE_TIMEOUT_MS = 30_000 (30 seconds for compile/build steps).
- DEFAULT_RUN_TIMEOUT_MS = 15_000 (15 seconds per test case).
- Containers run with --rm so they self-clean after exit.
- The runner sends SIGKILL to the docker process on timeout to ensure timely teardown.
- stdin is closed via Node's pipe so well-behaved programs receive EOF cleanly.

16.4 Compile-Once-Run-N Session

For Submit, the runner compiles the source one time and then runs the resulting binary against each test case. This collapses N compile+run cycles into 1 compile + N runs, materially reducing submission latency for compiled languages (C, C++, Java, C#).

16.5 Interactive Terminal

Run mode wires the container's stdin/stdout/stderr through a WebSocket to **xterm.js** on the client. InteractiveTerminal.jsx registers a custom link provider (term.registerLinkProvider) that matches common error-line patterns - line N, file.ext:N, language-specific stack traces - and turns them into clickable links. Clicking a link calls the parent's onErrorLineClick which jumps the Monaco editor to the matching line.

17. AI System Architecture

This section describes the AI Mentor's internal architecture. It is the single most differentiated component of the platform and is not adequately covered by a single 'send context to LLM' diagram.

17.1 Two-Stage Tutor / Student Loop

Mentor replies are produced by a two-stage pipeline:

64. Stage 1 - Mentor (qwen3-coder:30b) generates a draft reply from the assembled prompt.
65. Stage 2 - Validator audits the draft against the academic-integrity policy. If the draft passes, it is forwarded to the student; if it fails, it is either repaired or replaced with a safe fallback hint.

The split exists because mentor models are good at producing helpful text but bad at policing themselves. A small, fast validator is cheaper than expecting a large model to be its own judge and improves consistency under pressure (long sessions, exam-like prompts, or attempts to extract the solution).

17.2 Heuristic Floor + AI Escalation

The validator (backend/src/services/validator.ts) implements a two-tier check:

- Tier 1 - Heuristic floor. Cheap regex/pattern checks that catch the obvious violations without an LLM call: banned phrases (negation-aware), explicit exact-fix directives, runtime guessing, assignment walkthroughs, solution-seek leaks, overly long responses, and others. This tier short-circuits clear-cut rejections and is fast enough to run on every reply.
- Tier 2 - AI validator (qwen2.5:3b-instruct). Invoked only when the heuristic floor is undecided. Issues a JSON verdict (approve / repair / reject) plus a reason.

Heuristic floor highlights:

- Banned phrase detection is negation-aware via bannedPhraseIsAsserted() - the mentor saying 'I won't write the complete solution' does not trip the rule, but the mentor saying 'here is the complete solution' does.
- EXACT_FIX_PATTERNS uses a negative lookbehind to avoid false positives on noun-form 'change' (e.g. 'minimal change needed to fix' is fine; 'change line 14 to ...' is not).
- 8-second timeout on the AI validator with graceful fallback to the heuristic verdict if the small model is slow.

17.3 Mentor Quality Assessment

backend/src/services/mentorQuality.ts runs in parallel with the validator and flags low-quality replies even when they are policy-compliant. Detections include:

- transcript_artifact - the model leaked role labels or system text
- generic_fallback - the reply is a non-specific encouragement with no engagement
- echoes_question - the reply restates the student's question (Jaccard similarity >= 0.82)
- repeats_previous_reply - the reply is too similar to a prior mentor turn (Jaccard >= 0.72)

- ignores_error_context - the student's run output is non-empty but the reply does not reference it
- ignores_focused_line - the student provided cursor context but the reply does not address it

17.4 Mentor Prompt Composition

The mentor prompt is assembled in buildMentorPrompt() from these sections in order:

66. System role and global rules (no solutions, hint-level discipline, Socratic style).
67. Locale directive ('Respond in Turkish (Türkçe). Use natural Turkish prose; keep code, error messages, and API names in their original form.' when applicable).
68. Assignment text.
69. Current student code (all open files).
70. [EXECUTION CONTEXT] block summarising the last run: compile_error / runtime_error / wrong_answer / accepted, plus stdout/stderr.
71. [FOCUSED CODE NEAR CURSOR] block (when the student provides editor context): a +/-5 line window centred on the cursor with a '>' marker on the active line.
72. [CONVERSATION SO FAR] block: last 20 turns of chat, labelled Student/Mentor, with an instruction to build on prior context.
73. [STUDENT MESSAGE] - the current question.
74. Mode-specific override (HINT MODE includes the current hint level).

Ollama is called with num_ctx 16384 to fit prompt + history + reply comfortably; the validator uses num_ctx 4096.

17.5 Conversation Memory

The mentor does not store conversation state server-side per session. Instead, the frontend (ProblemPage.jsx) holds the chat log in React state and forwards a sanitised snapshot on every request:

- Last 20 entries (10 user/assistant turns).
- Excludes the greeting message and any still-streaming bubble.
- Excludes failed-request error bubbles ([error] ...).
- Sent as conversationHistory: [{role, content}].

This gives the mentor working memory across a session without requiring a stateful conversation store on the backend.

17.6 Hint-Level Progression

Hints escalate through discrete levels: a light nudge at level 1, a more pointed observation at level 2, a partial walkthrough at level 4, and an explicit non-code walkthrough at level 5. The hint level is tracked per problem on the frontend and is included in HINT MODE prompts so the mentor knows how much to reveal.

17.7 Locale Handling

detectMessageMode() and normalizeMentorLocale() select between English and Turkish based on the student's message and an explicit locale hint. Turkish locale activates: bilingual fallback messages, Turkish-language banned-phrase patterns, and a system directive that keeps code/errors/API names in their original form while switching the prose to Turkish.

17.8 Debug-Prints Shortcut

The student workspace exposes a Debug-Prints button next to Run/Submit. Clicking it: (1) captures the editor cursor and the +/-5 line window via getCursorContext(); (2) chooses a language-appropriate print call (printf, print, console.log, std::cout, System.out.println, Console.WriteLine); (3) sends a pre-written request to the mentor asking for a few temporary trace prints around the cursor line, with explicit instructions not to change the program logic; (4) the mentor returns a small snippet the student can paste back. This teaches print-debugging without the mentor giving away the answer.

17.9 Model Sizing and VRAM Budget

Both models are loaded concurrently in the L4's 23 GB VRAM:

Model	VRAM	Context	Role
qwen3-coder:30b	~19.4 GB	16384	Mentor
qwen2.5:3b-instruct	~2.4 GB	4096	Validator
Free	~1.2 GB	-	KV cache headroom

Models are pinned with OLLAMA_KEEP_ALIVE=-1 so they survive idle time. OLLAMA_FLASH_ATTENTION=1 is set to reduce KV cache memory. Cold start of the 30B model is approximately 30-60 seconds (disk-bound) and is hidden from users by a warm-up container that issues a one-token request to each model on Compose startup.

18. Security and Academic Integrity

The platform combines several mechanisms to improve security and academic integrity:

- Role-based authentication and authorization (JWT, per-route middleware).
- Separate student and teacher portals at the routing layer.
- Restricted Coordinator Teacher privileges (no infrastructure access).
- Hidden reference solutions (never sent to students).
- Hidden test cases (separated public/hidden flag).
- Exam start and end time gating, enforced server-side.
- AI Mentor route blocks calls during Exam Mode.
- Exam Mode frontend lockdown (single column, no AI panel, beforeunload + contextmenu guards).

- Dual enforcement: hiding UI is not enough; the backend blocks API calls too.
- Two-stage validator rejects mentor replies that leak solutions even when prompted aggressively.
- Submission history is immutable and timestamped.
- Teacher-controlled grading (AI suggestions never apply automatically).
- Teacher registration approval workflow.
- Docker isolation of every student execution.

Exam Mode is the most important academic-integrity feature: AI assistance is available during practice while disabled, hidden, and dual-enforced during exams.

19. Testing and Validation

The system is validated through functional tests covering the main workflows.

Student-side scenarios:

- Student login
- Assignment listing
- Problem selection
- Code editing (single and multi-file)
- Run operation (interactive terminal)
- Submit operation (compile-once-run-N)
- Compiler error display with clickable error-line links
- Submission history update
- AI Mentor hint request (validator approved)
- AI Mentor solution-seek attempt (validator rejected)
- AI Mentor restriction during Exam Mode (UI hidden + API blocked)
- Debug-Prints shortcut
- Tutorial browsing through the 3-level TOC
- Tutorial section PDF export
- Analytics display
- Flashcard review

Teacher-side scenarios:

- Teacher login
- Assignment creation (problem-first form auto-fill)
- Assignment editing
- Student enrollment (groups + individuals)
- Exam Mode activation
- Question creation
- Test case management (public/hidden)
- AI-generated question suggestion
- Rubric creation

- AI-generated rubric suggestion
- Submission review
- Rubric-based grading
- AI grading suggestion
- Grade export (Excel)
- Class analytics display

Coordinator Teacher scenarios:

- Viewing all students
- Assigning students to teachers
- Reviewing teacher registration requests
- Approving teacher registrations
- Rejecting teacher registrations

20. Challenges and Solutions

20.1 Balancing AI Support and Academic Integrity

Challenge: AI assistance is useful for learning, but unrestricted access during exams could harm academic integrity.

Solution: Two-stage AI Mentor (mentor + validator) prevents solution leaks even during normal learning. Exam Mode provides dual enforcement: the frontend hides the AI panel and locks the workspace, and the backend rejects mentor calls during the exam window.

20.2 Replacing the Code Execution Sandbox

Challenge: The original Judge0 backend depended on isolate v1, which is incompatible with the host's cgroup v2 - every submission failed with TLE regardless of correctness.

Solution: A new Docker-based runner (dockerRunner.ts) replaced Judge0. Each Run/Submit spawns an ephemeral container with --rm, per-language images, explicit timeouts, and a compile-once-run-N session model that collapses N compile+run cycles into 1 compile + N runs for compiled languages.

20.3 Supporting Different Teaching Responsibilities

Challenge: A single teacher role is not enough for all academic workflows because some instructors may need broader coordination privileges.

Solution: The platform includes a Coordinator Teacher role that can approve teacher registrations, view all students, and assign students to other teachers without being a full administrator.

20.4 Supporting Multiple Programming Languages

Challenge: Programming assignments may use different languages: C, C++, Python, Java, C#, or JavaScript.

Solution: The platform stores supported languages per problem, picks the matching Docker image at runtime, and the frontend renders language-specific starter files in the editor tab bar.

20.5 Making Feedback Reusable

Challenge: Students may forget compiler errors, AI explanations, and teacher feedback after a session.

Solution: Flashcards are generated from each session with strict post-filters (isValidCard, dedupeCards, hasGrounding) so every card quotes specific tokens from the student's own code, and explicit GOOD/BAD examples in the prompt anchor card quality.

20.6 Reducing Teacher Workload

Challenge: Creating problems, rubrics, and grading submissions requires significant time.

Solution: AI-assisted tools help teachers generate problem variations, rubric suggestions, and grading suggestions. Nothing is applied without explicit teacher approval.

20.7 Migrating the Tutorial Corpus

Challenge: The original tutorial schema (body + trailing code) could not represent w3schools-style interleaved text and code, and rendering quality suffered.

Solution: A block-based schema (text / code / syntax / note / goodtoknow / list) was introduced, ~97 C tutorials were migrated into it (with a 3-level category/sub-category/topic hierarchy), and TutorialModal renders both new and legacy schemas. Per-section PDF export was added via jsPDF.

21. Deployment Topology

The platform is deployed on a GCE virtual machine with an attached NVIDIA L4 GPU (23 GB VRAM, 15 GiB host RAM, no swap). All services run under **Docker Compose**:

Service	Image / runtime	Role
app-backend	Node + TypeScript (Express + Prisma)	REST + SSE API
app-frontend	Vite-built React static bundle	Web UI
postgres	Postgres	Persistent database
ollama	Ollama (GPU-enabled)	Local LLM serving
ollama-warmup	curl one-shot	Pre-warms both models on Compose up so first user does not wait for cold load

The Ollama service mounts the GPU via the NVIDIA container runtime (deploy.resources.reservations.devices or the shorter gpus: all form). Backend reaches Ollama over the internal Docker network at http://ollama:11434. Prisma migrations are applied non-interactively at backend container start. Compose volumes persist Postgres data and the Ollama model cache so neither has to be rebuilt across restarts.

22. Conclusion

The AI-Assisted Programming Platform for Education and Assessment provides a complete environment for programming education. It supports students with online code editing, interactive Docker-isolated execution, automated testing, a two-stage AI Mentor (qwen3-coder:30b mentor audited by qwen2.5:3b-instruct validator), a 97-tutorial block-based learning library, analytics, and feedback flashcards. It supports teachers with problem-first assignment management, question bank tools, group + individual enrollment, exam control, rubric-based grading, AI-assisted evaluation, and class analytics.

The system also includes a Coordinator Teacher role for broader academic management. This role can approve teacher registrations, view all students, and assign students to teachers, while still remaining separate from a full administrator role.

Overall, the implementation provides a practical balance between AI-supported learning and controlled assessment. AI is available during practice and homework to guide students through a Socratic, validator-audited mentor, while Exam Mode disables AI assistance, hides assistive panels, and gates submissions during exams to protect academic integrity.

Testing & Validation

1. Document Control

Field	Value
Project Name	AI-Assisted Programming Platform for Education and Assessment
Report Type	Testing and Validation Report
Version	1.6
Date	21.05.2026
System Under Test	Backend services, deployed web application, and CI/CD workflow
Repository Scope	Backend, frontend, infrastructure configuration, and deployed platform

2. Purpose

This report documents the testing and validation activities performed for the AI-Assisted Programming Platform for Education and Assessment. The goal is to verify the current functional state of the project through backend checks, deployed system checks, CI/CD review, bug tracking observations, and coverage-status review.

The report focuses on observable and reproducible validation evidence. It records both successful behavior and known quality gaps, including build failure, missing automated unit tests, and unavailable formal coverage metrics.

3. Scope

3.1 In Scope

- Backend health endpoint validation
- Student authentication validation
- Problem retrieval validation
- Student history validation
- AI hint endpoint validation
- Database migration validation
- Database seed validation
- Deployed student and teacher UI workflow validation
- CI/CD workflow review
- Bug tracking observations
- Coverage report availability check

3.2 Out of Scope

- Performance and load testing
- Penetration/security testing
- Cross-browser compatibility testing
- Accessibility certification

- Complete judged-output validation for all code execution cases

- Formal numeric coverage certification

4. Test Environment

Item	Configuration
Deployed Platform	https://aiassistedprogramming.com.tr/
Local Backend URL	http://127.0.0.1:5000
Operating System	Windows
Runtime	Node.js v24.14.1
Database	PostgreSQL 16
Container Runtime	Docker Desktop
Backend Port	5000
Judge0 Port	2358
AI Runtime Port	11434
Test Tools	PowerShell, Prisma CLI, Docker, deployed web UI, GitHub Actions workflow review

5. Test Design Specifications

Backend validation was performed by sending HTTP requests to local API endpoints and by running Prisma migration, seed, build, and test commands. System validation was performed on the deployed platform by checking student and teacher workflows. CI/CD validation was performed by reviewing the GitHub Actions workflow configuration. Bug tracking and coverage validation were handled by documenting observed failures and verifying whether automated test and coverage tooling existed.

The following test result statuses are used:

- Pass: The observed result matched the expected result.

- Fail: The observed result did not match the expected result.

- Partial Pass: The scenario was partially validated, but the collected evidence was not sufficient to confirm the full expected behavior.

5.1 Graphical User Interface (GUI)

5.1.1 Subfeatures to be tested

- Portal selection screen

- Student sign-in screen

- Teacher sign-in screen

- Student problem-solving screen

- Code run output panel

- AI Mentor chat panel

- Teacher dashboard

- Exam mode control panel

5.1.2 Test cases

ID	Requirement	Priority	Scenario Description	Expected Result	Actual Result	Status
GUI-01	Portal Selection	High	Open the deployed application landing page and verify portal selection options.	Student and Teacher portal entry points should be visible and accessible.	Both Student and Teacher portal options were displayed correctly.	Pass
GUI-02	Student Login Screen	High	Open the Student portal login page.	Student sign-in form should load correctly.	Student sign-in form loaded successfully.	Pass
GUI-03	Teacher Login Screen	High	Open the Teacher portal login page.	Teacher sign-in form should load correctly.	Teacher sign-in form loaded successfully.	Pass
GUI-04	Student Problem Solving Screen	High	Access a student problem page containing the editor and problem description.	Problem description, language selector, and code editor should be visible.	The student problem-solving screen displayed all expected components.	Pass
GUI-05	Code Run Flow	High	Trigger code execution from the student editor screen.	The output panel should update after the run action.	The output panel showed Compiling... and Compiled OK; final judged status was not	Partial Pass

					confirmed from the collected evidence.	
GUI-06	AI Mentor Interaction	Medium	Send a hint request through the deployed AI Mentor chat interface.	The system should return a mentor response to the student.	The chat interface returned an AI Mentor response successfully.	Pass
GUI-07	Teacher Dashboard	Medium	Open the teacher dashboard after sign-in.	Teacher metrics and management widgets should be visible.	Dashboard metrics, assignments section, and controls were displayed.	Pass
GUI-08	Exam Mode Control	Medium	Access the teacher exam mode section.	Teacher should be able to view exam mode controls.	Exam mode toggle and student/group scope controls were visible.	Pass

5.2 Backend and API

5.2.1 Subfeatures to be tested

- Backend health endpoint

- Student authentication endpoint

- Problem retrieval endpoint

- Student history endpoint

- AI hint endpoint

5.2.2 Test cases

ID	Requirement	Priority	Scenario Description	Expected Result	Actual Result	Status
API-01	Backend Availability	High	Send a request to the /health endpoint.	The API should respond successfully and indicate healthy service status.	The API returned status=ok and service=api.	Pass
API-02	Student Authentication	High	Submit valid student credentials to /api/auth/login.	The system should authenticate the student and return an access token.	Login succeeded and an accessToken was returned.	Pass
API-03	Problem Retrieval	High	Request /api/problems using a valid bearer token.	The system should return the available programming problems.	The endpoint returned the seeded problem list successfully.	Pass
API-04	Submission History	Medium	Request /api/student/history using a valid bearer token.	The system should return stored student submission	The endpoint returned submission history records successfully.	Pass

				records.		
API -05	AI Hint Response	Mediu m	Submit a request to /api/ai/hint with problem ID, language, and source code.	The system should return AI mentor guidance.	The endpoint returned success=true ; fallback response was used because mentor service was unavailable.	Pass

5.3 Database Integration

5.3.1 Subfeatures to be tested

- Prisma migration execution

- Seed data initialization

- PostgreSQL connectivity through Prisma

5.3.2 Test cases

ID	Requirement	Priority	Scenario Description	Expected Result	Actual Result	Status
DB-01	Database Migration	High	Run npx prisma migrate dev in the backend environment.	The database schema should be applied successfully.	Prisma reported that the schema was already in sync and no pending	Pass

					migration remained.	
DB-02	Database Seed	High	Run npm run prisma:seed in the backend environment.	Demo data should be inserted successfully.	Seed process completed successfully.	Pass

5.4 Build, Unit Testing, and CI/CD

5.4.1 Subfeatures to be tested

- Backend TypeScript build

- Backend unit test command

- GitHub Actions workflow configuration

5.4.2 Test cases

ID	Requirement	Priority	Scenario Description	Expected Result	Actual Result	Status
BUILD-01	Backend Build Validation	High	Run npm run build in the backend environment.	Backend TypeScript source should compile without errors.	Build failed due to TypeScript type mismatches in AI logging and audit-related	Fail

					files.	
UNIT-01	Unit Test Execution	Medium	Run npm test in the backend environment.	Automated unit tests should execute successfully.	No real unit test suite exists; placeholder script returned Error: no test specified.	Fail
CICD-01	CI/CD Workflow Review	Medium	Review .github/workflows/ci.yml.	CI should include dependency installation and build validation steps.	GitHub Actions workflow includes backend install, Prisma generate, backend build, frontend install, and frontend build steps. Automated test and coverage steps are not configured.	Partial Pass

5.5 Bug Tracking and Coverage

5.5.1 Subfeatures to be tested

- Known issue documentation

- Coverage report availability

- Validation gaps related to automated testing

5.5.2 Test cases

ID	Requirement	Priority	Scenario Description	Expected Result	Actual Result	Status
BUG-01	Bug Tracking	Medium	Review observed defects during validation.	Known failures should be documented with severity and current status.	Judge0 runtime issue, AI fallback dependency issue, backend build failure, missing unit tests, and coverage absence were documented as tracked quality issues.	Pass
COV-01	Coverage Report	Medium	Check whether formal coverage reporting is	The project should provide automated coverage	No coverage script or coverage publication step is	Fail

			available.	output if coverage is required.	currently configured. Formal numeric coverage is not available.	

6. Detailed Test Case Specifications

Section 5 lists the test cases at summary level. Its purpose is to show which requirement is tested, why it matters, and whether the result passed. This section expands the same validation scope into execution-ready detailed test cases. In other words, Section 5 answers "what is tested?", while Section 6 answers "how is the test executed and what evidence was observed?".

6.1 Summary Test Cases vs Detailed Test Cases

Comparison Area	Summary Test Cases in Section 5	Detailed Test Cases in Section 6
Main purpose	Requirement coverage and high-level result tracking	Repeatable execution procedure and evidence recording
Typical reader	Reviewer, instructor, project manager	Tester, developer, maintainer
Level of detail	Short scenario description with expected and actual result	Purpose, setup, test data, procedure, verification, cleanup, observed result, and status

Use in report	Shows the test matrix	Shows how each important test can be reproduced
Example field	Scenario Description	Procedure [A01] and Verification [V01]

6.2 Detailed Test Case Template

Each detailed test case follows the template below. TC means Test Case.

Field	Description
TC_ID	Unique identifier of the detailed test case
Related Summary ID	Matching test case ID from Section 5
Requirements	Functional or quality area being validated
Priority	Relative importance of the test case
Estimated Time Needed	Approximate time required to execute the test
Dependency	Required external state, service, account, or

	configuration
Purpose	What the test intends to prove
Setup	Required system state before execution
Test Data	Input values, credentials, commands, endpoint paths, or files used during the test
Procedure	[A01] Ordered actions performed by the tester
Verification	[V01] Result that should occur if the system behaves correctly
Observed Result	Result observed during validation
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass, Fail, or Partial Pass

6.3 Detailed Test Cases

GUI.PORTAL.01 Portal Selection

Field	Value
TC_ID	GUI.PORTAL.01
Related Summary ID	GUI-01
Requirements	Portal Selection
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that users can choose between Student and Teacher portals from the deployed landing page.
Setup	The deployed platform must be reachable from a browser.
Test Data	URL: https://aiassistedprogramming.com.tr/
Procedure	[A01] Open the deployed URL. Inspect the landing page. Confirm that Student Portal and Teacher Portal options are visible.

Verification	[V01] Both portal entry points should be displayed and selectable.
Observed Result	Student and Teacher portal options were displayed correctly.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

GUI.SLOGIN.01 Student Login Screen

Field	Value
TC_ID	GUI.SLOGIN.01
Related Summary ID	GUI-02
Requirements	Student Login Screen
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.

Purpose	Verify that the student sign-in screen loads correctly.
Setup	The deployed platform must be reachable and the Student portal must be selectable.
Test Data	Demo student account shown on the UI: student1@demo.com
Procedure	[A01] Open the landing page. Select Student Portal. Inspect the login form fields and sign-in action.
Verification	[V01] Student email field, password field, and sign-in button should be visible.
Observed Result	Student sign-in form loaded successfully.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

GUI.TLOGIN.01 Teacher Login Screen

Field	Value
TC_ID	GUI.TLOGIN.01
Related Summary ID	GUI-03
Requirements	Teacher Login Screen
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that the teacher sign-in screen loads correctly.
Setup	The deployed platform must be reachable and the Teacher portal must be selectable.
Test Data	Demo teacher account shown on the UI: teacher1@demo.com
Procedure	[A01] Open the landing page. Select Teacher Portal. Inspect the

	login form fields and sign-in action.
Verification	[V01] Teacher email field, password field, and sign-in button should be visible.
Observed Result	Teacher sign-in form loaded successfully.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

GUI.PROBLEM.01 Student Problem Solving Screen

Field	Value
TC_ID	GUI.PROBLEM.01
Related Summary ID	GUI-04
Requirements	Student Problem Solving Screen
Priority	High

Estimated Time Needed	3 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that the student can access the assignment workspace and see the required problem-solving components.
Setup	Student portal must be accessible and at least one assignment must exist.
Test Data	Problem example: Check Prime Number
Procedure	[A01] Sign in as student. Open an assignment. Inspect the problem statement, language selector, code editor, Run button, Submit button, and AI Mentor panel.
Verification	[V01] Problem description, editor, language selector, run controls, and mentor panel should be

	visible.
Observed Result	Problem description, language selector, editor, and mentor panel were visible.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

GUI.RUN.01 Code Run Flow

Field	Value
TC_ID	GUI.RUN.01
Related Summary ID	GUI-05
Requirements	Code Run Flow
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Required service, page, credentials, or configuration must be

	available before execution.
Purpose	Verify that the student code editor can start a code execution attempt and update the output panel.
Setup	Student problem-solving page must be open and a language must be selected.
Test Data	Language: C. Starter code from the Check Prime Number problem.
Procedure	[A01] Select the language. Keep or edit the starter code. Click Run. Observe the output panel.
Verification	[V01] Output panel should display compilation or execution feedback after the Run action.
Observed Result	Output panel showed Compiling... and Compiled OK. Final judged result was not captured in the collected evidence.

Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Partial Pass

GUI.MENTOR.01 AI Mentor Interaction

Field	Value
TC_ID	GUI.MENTOR.01
Related Summary ID	GUI-06
Requirements	AI Mentor Interaction
Priority	Medium
Estimated Time Needed	2 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that the AI Mentor chat panel can return a response to a

	student hint request.
Setup	Student problem page and AI Mentor panel must be visible.
Test Data	Prompt example: Give me a hint
Procedure	[A01] Open the AI Mentor chat panel. Submit a hint request. Observe whether a mentor reply is displayed.
Verification	[V01] AI Mentor should return a contextual response.
Observed Result	AI Mentor returned a visible response in the chat panel.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

GUI.TDASH.01 Teacher Dashboard

Field	Value
TC_ID	GUI.TDASH.01
Related Summary ID	GUI-07
Requirements	Teacher Dashboard
Priority	Medium
Estimated Time Needed	2 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that the teacher dashboard displays class-level metrics and management sections.
Setup	Teacher portal must be accessible and teacher credentials must be valid.
Test Data	Demo teacher account: teacher1@demo.com
Procedure	[A01] Sign in as teacher. Open the dashboard page. Inspect class overview,

	assignments, and dashboard controls.
Verification	[V01] Teacher metrics and management widgets should be visible.
Observed Result	Dashboard metrics, assignments section, and controls were displayed.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

GUI.EXAM.01 Exam Mode Control

Field	Value
TC_ID	GUI.EXAM.01
Related Summary ID	GUI-08
Requirements	Exam Mode Control
Priority	Medium

Estimated Time Needed	2 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that teacher exam mode controls are visible.
Setup	Teacher dashboard must be open.
Test Data	Scope options: all students and visible student groups
Procedure	[A01] Open the teacher dashboard. Inspect the Exam Mode section. Check whether toggle and scope controls are visible.
Verification	[V01] Exam mode toggle and student or group targeting controls should be visible.
Observed Result	Exam mode toggle and group selection controls were visible.

Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

API.HEALTH.01 Backend Availability

Field	Value
TC_ID	API.HEALTH.01
Related Summary ID	API-01
Requirements	Backend Availability
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that the backend service is reachable.
Setup	Backend must be running

	locally on port 5000.
Test Data	GET http://127.0.0.1:5000/health
Procedure	[A01] Start the backend service. Send a GET request to the health endpoint using PowerShell. Record the response.
Verification	[V01] API should return a healthy status response.
Observed Result	API returned status=ok and service=api.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

API.AUTH.01 Student Authentication

Field	Value
TC_ID	API.AUTH.01
Related Summary ID	API-02

Requirements	Student Authentication
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that valid student credentials produce an authentication token.
Setup	Backend and database must be running. Seed data must exist.
Test Data	Email: student1@demo.com. Password: 123456. Endpoint: POST /api/auth/login
Procedure	[A01] Create a JSON request body with valid credentials. Send POST request to /api/auth/login. Inspect response body.
Verification	[V01] API should return an access token.

Observed Result	Login succeeded and accessToken was returned.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

API.PROBLEM.01 Problem Retrieval

Field	Value
TC_ID	API.PROBLEM.01
Related Summary ID	API-03
Requirements	Problem Retrieval
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.

Purpose	Verify that authenticated students can retrieve programming problems.
Setup	Backend must be running and a valid bearer token must be available.
Test Data	GET /api/problems with Authorization header
Procedure	[A01] Authenticate as student. Store the returned token. Send GET request to /api/problems using the bearer token.
Verification	[V01] API should return seeded problem records.
Observed Result	Endpoint returned seeded problem list successfully.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

API.HISTORY.01 Submission History

Field	Value
TC_ID	API.HISTORY.01
Related Summary ID	API-04
Requirements	Submission History
Priority	Medium
Estimated Time Needed	2 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that authenticated students can retrieve previous submission records.
Setup	Backend must be running, a valid bearer token must exist, and submission history must be present.
Test Data	GET /api/student/history

	with Authorization header
Procedure	[A01] Authenticate as student. Send GET request to /api/student/history. Inspect returned data collection.
Verification	[V01] API should return stored student submission history records.
Observed Result	Endpoint returned student submission history records successfully.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

API.HINT.01 AI Hint Response

Field	Value
TC_ID	API.HINT.01

Related Summary ID	API-05
Requirements	AI Hint Response
Priority	Medium
Estimated Time Needed	2 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that the backend AI hint endpoint returns guidance for a valid request.
Setup	Backend must be running and a valid bearer token must be available.
Test Data	problemId: 1. mode: hint. language: python. question: Bu cozum dogru mu?
Procedure	[A01] Authenticate as student. Build the JSON request body. Send POST request to /api/ai/hint. Record success, mentorReply, fallbackUsed, validator, and

	policyAction fields.
Verification	[V01] API should return mentor guidance and policy validation result.
Observed Result	Endpoint returned success=true. Fallback response was used because mentor service was unavailable.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

DB.MIGRATE.01 Database Migration

Field	Value
TC_ID	DB.MIGRATE.01
Related Summary ID	DB-01
Requirements	Database Migration
Priority	High

Estimated Time Needed	3 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that the Prisma database schema can be applied successfully.
Setup	PostgreSQL container must be running and DATABASE_URL must be configured.
Test Data	Command: npx prisma migrate dev
Procedure	[A01] Open terminal in backend directory. Run the Prisma migration command. Observe migration result.
Verification	[V01] Database schema should be applied or reported as already synchronized.
Observed Result	Prisma reported that the schema was already in sync and no pending migration remained.

Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

DB.SEED.01 Database Seed

Field	Value
TC_ID	DB.SEED.01
Related Summary ID	DB-02
Requirements	Database Seed
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that demo data can be inserted into the database.

Setup	PostgreSQL container must be running and Prisma client must be generated.
Test Data	Command: npm run prisma:seed
Procedure	[A01] Open terminal in backend directory. Run the seed command. Observe terminal output.
Verification	[V01] Seed data should be inserted successfully.
Observed Result	Seed process completed successfully.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

BUILD.TSC.01 Backend Build Validation

Field	Value

TC_ID	BUILD.TSC.01
Related Summary ID	BUILD-01
Requirements	Backend Build Validation
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify whether the backend TypeScript project compiles successfully.
Setup	Backend dependencies must be installed.
Test Data	Command: npm run build
Procedure	[A01] Open terminal in backend directory. Run npm run build. Record compiler output.
Verification	[V01] TypeScript compilation should

	finish without errors.
Observed Result	Build failed due to TypeScript type mismatches in AI logging and audit-related files.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Fail

UNIT.NPM.01 Unit Test Execution

Field	Value
TC_ID	UNIT.NPM.01
Related Summary ID	UNIT-01
Requirements	Unit Test Execution
Priority	Medium
Estimated Time Needed	2 Minutes
Dependency	Required service, page, credentials, or

	configuration must be available before execution.
Purpose	Verify whether an automated backend unit test suite exists and can be executed.
Setup	Backend dependencies must be installed.
Test Data	Command: npm test
Procedure	[A01] Open terminal in backend directory. Run npm test. Record the returned output and exit status.
Verification	[V01] Automated unit tests should execute successfully.
Observed Result	Placeholder script returned Error: no test specified.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Fail

CICD.WORKFLOW.01 CI/CD Workflow Review

Field	Value
TC_ID	CICD.WORKFLOW.01
Related Summary ID	CICD-01
Requirements	CI/CD Workflow Review
Priority	Medium
Estimated Time Needed	2 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that the repository contains automated build validation workflow steps.
Setup	Repository workflow configuration must be available.
Test Data	File: .github/workflows/ci.yml
Procedure	[A01] Open the CI workflow file. Review

	backend install, Prisma generate, backend build, frontend install, frontend build, test, and coverage steps.
Verification	[V01] Workflow should include dependency installation, build validation, test execution, and coverage reporting if required.
Observed Result	Workflow includes install, Prisma generate, backend build, frontend install, and frontend build. Automated test and coverage steps are not configured.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Partial Pass

BUG.TRACK.01 Bug Tracking

Field	Value
TC_ID	BUG.TRACK.01

Related Summary ID	BUG-01
Requirements	Bug Tracking
Priority	Medium
Estimated Time Needed	2 Minutes
Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify that known defects discovered during validation are recorded in the report.
Setup	Validation output and observed failures must be available.
Test Data	Judge0 runtime issue, AI fallback dependency issue, backend build failure, missing unit tests, missing coverage report
Procedure	[A01] Review validation results. List observed issues. Assign each issue a current status in the report.

Verification	[V01] Known issues should be documented clearly with current validation impact.
Observed Result	Known issues were documented as tracked quality issues.
Cleanup	Close the related page, clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Pass

COV.REPORT.01 Coverage Report

Field	Value
TC_ID	COV.REPORT.01
Related Summary ID	COV-01
Requirements	Coverage Report
Priority	Medium
Estimated Time Needed	2 Minutes

Dependency	Required service, page, credentials, or configuration must be available before execution.
Purpose	Verify whether formal automated code coverage output is available.
Setup	Repository package scripts and CI workflow must be available.
Test Data	Backend package scripts and CI workflow configuration
Procedure	[A01] Inspect backend test scripts. Inspect CI workflow for coverage generation or publication steps.
Verification	[V01] Coverage report should be available if coverage reporting is configured.
Observed Result	No coverage script or coverage publication step was found. Formal numeric coverage is not available.
Cleanup	Close the related page,

	clear temporary request variables if needed, and return the terminal or browser to idle state.
Status	Fail

Conclusion & Future Work

Overall Achievement

The project successfully evolved into a functional full-stack educational platform that combines programming practice, automated code execution, AI-assisted learning, teacher-controlled assessment, structured grading, analytics, and academic integrity mechanisms within a single system. Instead of remaining a simple assignment submission tool, the platform provides separate workflows for students, teachers, and coordinator-level academic management.

One of the main achievements of the project is the integration of AI as a controlled educational assistant rather than an unrestricted problem solver. On the student side, the AI Mentor supports learning by giving hints, explaining errors, and guiding students through programming concepts. On the teacher side, AI is used to reduce workload through problem variation generation, rubric generation, and grading suggestions. This dual use of AI supports both learning and assessment while keeping the teacher in control of final academic decisions.

The system also achieved a strong connection between its major technical components. The frontend provides role-based interfaces and an IDE-like coding workspace, while the backend handles authentication, authorization, assignment logic, code execution requests, AI service communication, grading, analytics, and coordinator-level operations. The database supports these workflows by storing users, problems, test cases, assignments, submissions, rubrics, grades, AI interactions, flashcards, and exam-related records. Together, these components form a coherent platform rather than a set of disconnected features.

Another important achievement is the inclusion of academic integrity mechanisms. Features such as hidden test cases, hidden reference solutions, teacher-controlled Exam Mode, submission history, role-based authorization, and teacher registration approval help the platform support both practice-oriented learning and controlled assessment scenarios. This balance is especially important because the project aims to use AI in education without allowing it to replace student problem solving.

Overall, the project achieved more than the minimum expected scope. It demonstrates a working prototype of an AI-assisted programming education and assessment platform, with meaningful support for students, teachers, and academic coordinators. The current implementation provides a solid foundation for future improvements in production security, scalability, AI safety validation, and real classroom evaluation.

Key Achievements

The project achieved several important outcomes across the frontend, backend, AI, database, and deployment layers. These achievements show that the platform became more than a

basic programming assignment system and developed into an integrated educational environment.

One of the main achievements is the creation of a role-based learning and assessment workflow. Students, teachers, and the Coordinator Teacher have separate interfaces and permissions according to their responsibilities. Students can solve programming problems, run and submit code, receive feedback, use AI Mentor support, review tutorials, track progress, and study with flashcards. Teachers can manage assignments, enroll students, create and organize questions, review submissions, generate rubrics, use AI-assisted grading suggestions, and analyze class performance. The Coordinator Teacher role extends the system by supporting teacher approval and student-to-teacher assignment workflows.

Another achievement is the successful integration of browser-based programming practice with automated code execution. The platform allows students to write code directly in the browser, execute it, see compiler or runtime feedback, and submit solutions for evaluation. This makes the system closer to an educational online judge while still keeping the user experience focused on learning rather than only pass/fail results.

The AI integration is also a major achievement of the project. Instead of using AI as a direct answer generator, the platform uses AI as a controlled mentor for students and as a productivity assistant for teachers. The AI Mentor can guide students through hints, explanations, debugging support, and concept clarification. On the teacher side, AI supports question variation generation, rubric generation, and grading suggestions. This design supports the original goal of using AI to improve education while still keeping students and teachers responsible for the learning and assessment process.

The project also achieved a strong level of system integration. The frontend, backend, database, code execution layer, and AI services work together through structured workflows. The frontend provides role-specific user experiences, the backend manages authentication, authorization, business logic, AI communication, execution requests, grading, and analytics, while the database stores the core educational data such as users, problems, assignments, test cases, submissions, rubrics, grades, AI interactions, flashcards, and exam-related records.

Academic integrity is another important achievement. The platform includes mechanisms such as hidden test cases, hidden reference solutions, role-based authorization, submission history, teacher registration approval, and Exam Mode. Exam Mode is especially important because it allows the same platform to support two different educational contexts: AI-supported learning during practice and homework, and restricted AI access during exams.

Finally, the project reached a deployable prototype level through Docker-based and cloud-oriented infrastructure. This is important because the system was not only implemented as isolated local components, but also organized in a way that can be demonstrated and

extended as a full-stack application. Although further production hardening is still needed, the current implementation provides a strong foundation for future institutional use.

Limitations

Although the project reached a functional and integrated prototype level, several limitations remain before the platform can be considered fully ready for large-scale institutional or production use. These limitations do not reduce the value of the current implementation, but they show the areas that require further validation, hardening, and refinement.

One important limitation is related to production readiness. The current system demonstrates that the frontend, backend, database, AI services, and code execution layer can work together as a complete platform. However, production deployment would require stronger environment configuration, database backup and recovery strategies, monitoring, logging, HTTPS configuration, and stricter separation between development and production settings. Without these improvements, the system is more suitable as a strong prototype or demonstration environment than as a fully production-ready institutional service.

Security and authorization also remain important areas for improvement. The implementation includes role-based access control, hidden reference solutions, hidden test cases, Exam Mode, teacher approval, and student-teacher assignment control as academic integrity mechanisms. However, these mechanisms must be consistently enforced on the backend side, not only through frontend visibility or interface restrictions. Sensitive data such as reference solutions, hidden tests, grades, AI logs, and student submissions must be protected with strict ownership checks and carefully reviewed API responses.

Another limitation is the reliability and safety of AI assistance. The original project proposal already identified the risk that AI may accidentally provide complete answers instead of guidance. The current system addresses this risk through the idea of AI as a mentor rather than a solver, but this still requires continuous testing and validation. AI responses should be checked to ensure that they provide hints, explanations, and learning support without directly solving assignments. This is especially important in programming education, where even a small code suggestion can sometimes reveal the full solution.

The Exam Mode mechanism is a strong academic integrity feature, but it should also be seen as a limitation if used alone. The implementation correctly disables AI Mentor access during exam scenarios, allowing the platform to separate learning mode from assessment mode. However, real exam integrity requires more than disabling AI. Backend enforcement, activity logging, violation tracking, secure submission rules, and careful review of student behavior would be necessary for higher-stakes assessment environments.

Scalability is another limitation. The platform includes resource-heavy components such as browser-based code editing, automated code execution, AI requests, analytics, and teacher dashboards. The proposal also identified scalability for multiple simultaneous classroom or exam users as a challenging issue. Therefore, before real classroom deployment, the system

should be tested under concurrent users, simultaneous code executions, and multiple AI requests. Rate limiting, request queues, execution sandbox limits, and performance monitoring would be important future improvements.

The project also has limitations in terms of evaluation. The implementation report lists functional validation scenarios for student, teacher, and coordinator workflows, such as login, assignment creation, code execution, AI Mentor restriction, grading, analytics, and teacher approval. However, functional testing alone does not fully measure educational effectiveness. The proposal mentions usability tests, student learning outcomes, time-to-solve comparisons, teacher workload reduction, and user satisfaction surveys as possible evaluation methods. These types of user-centered and educational evaluations would be needed to prove the platform's real impact in a classroom setting.

Finally, some limitations are related to maintainability and long-term extensibility. As the project expanded, the frontend, backend, AI services, database models, grading workflows, analytics, and exam features became more complex. This complexity is expected for a full-stack educational platform, but future development would benefit from more automated tests, clearer API contracts, shared TypeScript/database types, stronger documentation, and modular refactoring of large frontend and backend components.

Overall, the current limitations mainly reflect the difference between a successful academic prototype and a production-ready educational platform. The system already demonstrates the core concept effectively, but future work should focus on security hardening, AI safety validation, scalability testing, deployment reliability, maintainability, and real classroom evaluation.

Lessons Learned

One of the most important lessons learned from this project is that integrating AI into an educational platform is not only a model integration problem. The AI component becomes useful only when it is supported by proper context management, role-based access control, prompt design, validation, logging, and user interface design. In this project, AI was designed as a mentor for students and as a productivity assistant for teachers, rather than as an autonomous solver or grader. This showed that the educational value of AI depends strongly on how the surrounding system controls and guides its behavior.

Another key lesson is that academic integrity must be considered as an architectural requirement from the beginning. Since the platform allows students to receive AI support during programming practice, the system also needs mechanisms to prevent misuse during assessment. Features such as Exam Mode, hidden test cases, hidden reference solutions, teacher-controlled assignments, submission history, and role-based access control were therefore essential parts of the system design. The implementation report also emphasizes Exam Mode as a way to separate learning scenarios from exam scenarios, where AI Mentor access is disabled to preserve academic integrity.

The project also showed the importance of designing AI as a human-in-the-loop tool. On the teacher side, AI can generate problem variations, rubrics, and grading suggestions, but the final academic decision must remain under teacher control. This is important because AI-generated outputs can be useful, but they still require human review for correctness, fairness, and suitability. The proposal also identified risks such as AI accidentally providing complete answers and the need to minimize AI bias in evaluation, which confirms that AI support should be controlled and reviewed rather than blindly trusted.

Another lesson learned is that full-stack integration is more difficult than implementing isolated features. A feature can look complete on the frontend, but it also needs correct backend authorization, database support, API consistency, and security checks. For example, assignment management, code execution, AI Mentor support, grading, analytics, and exam rules all depend on multiple layers working together. This made it clear that a successful educational platform requires coherent communication between frontend components, backend services, database models, execution systems, and AI services.

The project also highlighted the importance of balancing usability with control. Students need a simple and supportive coding environment, while teachers need powerful management tools for assignments, grading, rubrics, question banks, and analytics. At the same time, the system must prevent students from accessing teacher-only data, hidden tests, reference solutions, or AI support during restricted exam periods. This balance between ease of use and strict control became one of the central design challenges of the platform.

Another important lesson is that deployment and infrastructure decisions affect the quality of the whole system. Docker-based organization and cloud-oriented deployment made it possible to run the platform as a connected full-stack application rather than separate local modules. However, this also showed that production readiness requires additional concerns such as environment configuration, database backup, HTTPS, monitoring, logging, and resource limits for code execution and AI requests.

The project also demonstrated that educational effectiveness cannot be proven only by implementing features. Although the platform provides AI Mentor support, analytics, feedback flashcards, grading tools, and teacher dashboards, their real value should eventually be measured through user testing. The original proposal mentioned evaluation methods such as usability tests, learning outcome comparison, teacher workload reduction, and user satisfaction surveys. This showed that future validation should include not only technical testing, but also educational and user-centered evaluation.

Overall, the main lesson learned is that building an AI-assisted programming education platform requires both technical and pedagogical thinking. The project combined software engineering, AI integration, database design, security, cloud deployment, and educational assessment workflows. This made the system more complex than a standard programming

assignment platform, but it also made the final prototype more meaningful as a foundation for future classroom-oriented development.

Future Work

Although the current system provides a strong foundation for an AI-assisted programming education and assessment platform, several improvements can be made in future work to increase its reliability, scalability, security, and educational effectiveness.

One important future direction is production hardening. The current prototype demonstrates that the frontend, backend, database, AI services, and code execution layer can work together as an integrated system. However, before the platform is used in a real institutional environment, deployment-related improvements should be completed. These include configuring a proper production environment, enabling HTTPS, improving reverse proxy settings, separating development and production configurations, adding monitoring and logging tools, and creating a reliable database backup and recovery strategy.

Another major future work area is security and authorization improvement. Since the platform stores sensitive educational data such as submissions, grades, hidden test cases, reference solutions, AI interaction logs, and student analytics, all access control rules should be reviewed carefully. Backend-side ownership checks should be strengthened so that users can only access the data they are authorized to view. Teacher-only data, hidden reference solutions, and hidden test cases should never be exposed to students through API responses. This is especially important because the implementation already relies on academic integrity mechanisms such as role-based authorization, hidden test cases, hidden reference solutions, and Exam Mode.

AI safety validation is another important future direction. The project was designed around the idea that AI should guide students without directly solving problems for them. The original proposal also identified the risk that AI may accidentally provide complete answers. Therefore, future work should include systematic testing of AI Mentor responses. A test set of programming questions, student mistakes, and AI prompts can be created to measure whether the AI gives hints, explanations, and conceptual guidance instead of full solutions. The validation and policy layers can also be improved to better detect direct answer leakage, overly specific code suggestions, and unsafe grading explanations.

Exam integrity can also be improved further. The current system includes Exam Mode, which disables AI Mentor access during exam-style assignments. This is a strong starting point, but higher-stakes exams require stronger protection. Future work can include stricter backend enforcement of exam rules, more detailed violation logging, improved fullscreen and tab-switch detection, secure exam session tracking, and clearer teacher dashboards for reviewing suspicious activity. In this way, Exam Mode can evolve from a simple AI restriction feature into a more complete assessment integrity module.

Scalability and performance should also be tested in future work. The platform includes resource-heavy components such as browser-based code editing, Docker-based code execution, AI requests, analytics dashboards, and real-time terminal features. Since the proposal identified scalability for multiple simultaneous classroom or exam users as a challenge, future work should include load testing with concurrent students, simultaneous submissions, and multiple AI requests. Based on these results, the system can be improved with request queues, rate limiting, execution time limits, container resource limits, caching, and frontend bundle optimization.

Another future direction is automated testing and quality assurance. The implementation report lists many functional validation scenarios for student, teacher, and coordinator workflows, including login, assignment creation, code execution, AI Mentor restriction, grading, analytics, and teacher approval. In future work, these scenarios can be converted into automated unit tests, integration tests, and end-to-end tests. This would make the system easier to maintain as new features are added and would reduce the risk of breaking important workflows.

Educational evaluation is also necessary for future development. The current platform includes many learning-oriented features such as AI Mentor support, tutorials, flashcards, analytics, and feedback-based study materials. However, the real educational value of these features should be measured with students and teachers. The original proposal suggested usability tests, learning outcome comparisons, time-to-solve measurements, teacher workload reduction metrics, and user satisfaction surveys. These evaluation methods can be used in future work to understand whether the platform actually improves learning, reduces teacher workload, and supports fair assessment.

The platform can also be extended with more advanced educational features. Future versions may include personalized problem recommendations, adaptive learning paths, misconception detection, plagiarism or code similarity analysis, richer teacher analytics, peer review workflows, and integration with learning management systems such as Moodle or Blackboard. These extensions would make the system more useful in real classroom and department-level scenarios.

Finally, maintainability can be improved as the system grows. Some frontend, backend, AI service, and database components may become harder to manage as more features are added. Future work should therefore include modular refactoring, clearer API documentation, shared type definitions, better database schema documentation, and more consistent error handling. These improvements would make the platform easier to develop, test, deploy, and extend in future semesters.

Overall, future work should focus not only on adding new features, but also on making the existing system more secure, reliable, scalable, maintainable, and educationally validated.

This would help transform the current successful prototype into a stronger platform that can be used in real programming courses and assessment environments.

Final Remarks

The AI-Assisted Programming Platform for Education and Assessment demonstrates that AI can be integrated into programming education in a controlled and meaningful way when it is supported by proper system design, role-based workflows, academic integrity mechanisms, and teacher supervision. The project successfully combines student learning support, teacher productivity tools, automated code execution, structured grading, analytics, and coordinator-level academic management in a single full-stack prototype.

At the same time, the project also shows that building an educational AI platform requires more than implementing features. Security, fairness, scalability, AI safety, deployment reliability, and real classroom validation are essential for turning the prototype into a production-ready system. Therefore, the current implementation should be seen as a strong foundation that proves the concept and provides a clear direction for future improvement.

Overall, the project achieved its main goal of creating an AI-assisted programming platform that supports learning without replacing student problem solving. With further work on production hardening, systematic testing, AI validation, and classroom-based evaluation, the platform has the potential to become a practical and reliable tool for programming education and assessment.